

CS5340

Uncertainty Modeling in AI

Lecture 5: Factor Graph and the Junction Tree Algorithm

Assoc. Prof. Lee Gim Hee

AY 2026/27

Semester 1

Course Schedule

Week	Date	Topic	Remarks
1	10 Aug	National Day Holiday	No Lecture
2	17 Aug	Introduction to probabilistic reasoning	Assignment 0: Python Numpy Tutorial (Ungraded)
	22 Aug	Bayesian networks (Directed graphical models)	Make-up Lecture: Sat, 9.30am-12.30pm (LT15, Recording will be provided)
3	24 Aug	Markov random Fields (Undirected graphical models)	
4	31 Aug	Variable elimination and belief propagation	Assignment 1: Belief propagation and maximal probability (15%) Assignment 1: Due (13 Sep, 2359 HRS)
	05 Sept	Factor graph and the junction tree algorithm	Make-up Lecture: Sat, 9.30am-12.30pm (LT15, Recording will be provided)
5	07 Sep	I will be traveling	No Lecture
6	14 Sep	Parameter learning with complete data	Assignment 2: Junction tree and parameter learning (15%) Assignment 2: Due (27 Sep, 2359 HRS)
-	21 Sep	Recess week	No Lecture
7	28 Sep	Mixture models and the EM algorithm	
8	05 Oct	Hidden Markov Models (HMM)	Assignment 3: Hidden Markov model (15%) Assignment 3: Due (18 Oct, 2359 HRS)
9	12 Oct	Monte Carlo inference (Sampling)	
10	19 Oct	Variational inference	Assignment 4: MCMC Sampling (15%) Assignment 4: Due (01 Nov, 2359 HRS)
11	26 Oct	Variational Autoencoder and Diffusion Models	
12	02 Nov	Graph-cut and alpha expansion	
13	09 Nov	Diwali/Deepavali	No Lecture

Final Exam: 21 Nov 2026

Acknowledgements

- A lot of slides and content of this lecture are adopted from:
 1. Michael I. Jordan "An introduction to probabilistic graphical models", 2002. Chapters 4.2, 4.3 and 17
<http://people.eecs.berkeley.edu/~jordan/prelims/chapter4.pdf>
<http://people.eecs.berkeley.edu/~jordan/prelims/chapter17.pdf>
 2. Daphne Koller and Nir Friedman, "Probabilistic graphical models" Chapter 10
 3. David Barber, "Bayesian reasoning and machine learning" Chapter 6
 4. Kevin Murphy, "Machine learning: a probabilistic approach" Chapter 20.4
 5. Christopher Bishop "Machine learning and pattern recognition" Chapter 8.4.3

Learning Outcomes

- Students should be able to:
 1. Represent a joint distribution with a **factor graph**, and use it to compute the marginal/conditional probabilities.
 2. Use the **max-product algorithm** to find the maximal probability and its configurations.
 3. Convert a DGM/UGM into the **junction tree** and use it to compute the marginal/conditional probabilities.

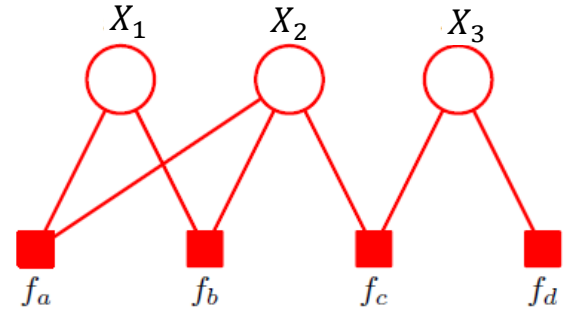
Factor Graphs

- **DGMs and UGMs**: allow a global function of several variables to be expressed as a **product of factors** over subsets of those variables.
- **Factor graphs** make this decomposition explicit by introducing **additional nodes for the factors** in addition to the nodes representing the variables.
- Unlike DGMs and UGMs, factor graphs are **NOT** designed for **conditional independence**, but for **more explicit** details of the **factorization**.

Factor Graphs: Graphical Representation

- A factor graph is a **bipartite graph**:

$$\mathcal{G}(\mathcal{V}, \mathcal{F}, \mathcal{E}),$$



where

- vertices** $\mathcal{V} \in \{X_1, \dots, X_n\}$: index the random variables,
 - vertices** $\mathcal{F} \in \{\dots, f_s, \dots\}$: index the factors and
 - undirected edges** $\mathcal{E}$: link each factor node f_s to all variable nodes X_s that f_s depends.
-
- We use **round nodes** to represent random variables and **square nodes** to represent factors.

Image source: "Pattern recognition and machine learning", Christopher Bishop

Factor Graphs: Joint Distribution

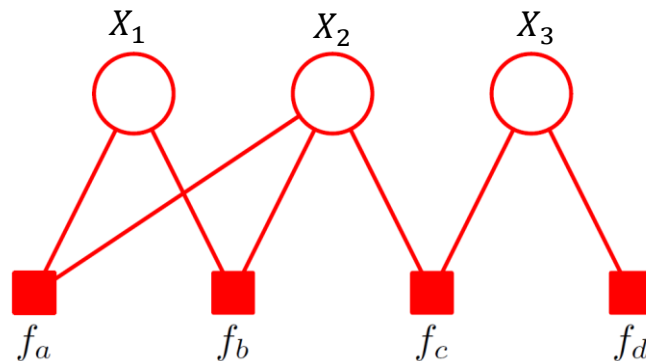
- We write the **joint distribution** over a set of variables in the form of a **product of factors**:

$$p(\mathbf{x}) = \prod_s f_s(\mathbf{x}_s)$$

- Where X_s denotes a **subset of the variables** $X \in \{X_1, \dots, X_n\}$.
- Each **factor** f_s is a function of a corresponding set of variables X_s .

Factor Graphs

Example:



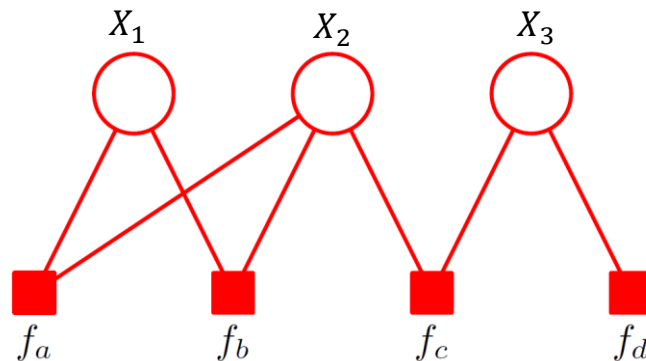
$$p(\mathbf{x}) = f_a(x_1, x_2) f_b(x_1, x_2) f_c(x_2, x_3) f_d(x_3)$$

- Note that there are two factors $f_a(x_1, x_2)$ and $f_b(x_1, x_2)$ that are defined over the **same set of variables**.
- In an **undirected graph**, product of two such factors would simply be **lumped together** into the same clique potential.

Image source: "Pattern recognition and machine learning", Christopher Bishop

Factor Graphs

Example:



$$p(\mathbf{x}) = f_a(x_1, x_2) f_b(x_1, x_2) f_c(x_2, x_3) f_d(x_3)$$

- Similarly, $f_c(x_2, x_3)$ and $f_d(x_3)$ could be combined into a single potential over X_2 and X_3 .
- The factor graph **keeps such factors explicit**, so is able to convey more detailed information about the underlying factorization.

Image source: "Pattern recognition and machine learning", Christopher Bishop

Convert DGM to Factor Graph

- Recall the **factorization of DGMs** is defined as:

$$p(x_1, \dots, x_N) = \prod_{i=1}^N p(x_i | x_{\pi_i})$$

- Convert a DGM into a factor graph by representing the **local conditional distributions** $p(x_i | x_{\pi_i})$ as **factors** $f_s(x_s)$.

Convert UGM to Factor Graph

- Recall the **factorization of UGMs** is defined as:

$$p(\mathbf{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{y}_c|\boldsymbol{\theta}_c)$$

- Convert a UGM into a factor graph by representing the **potential functions over the maximal cliques as factors** $f_s(\mathbf{x}_s)$.
- Normalizing coefficient** $1/Z$ can be viewed as a factor defined over the **empty set of variables**.

DGM/UGM to Factor Graph

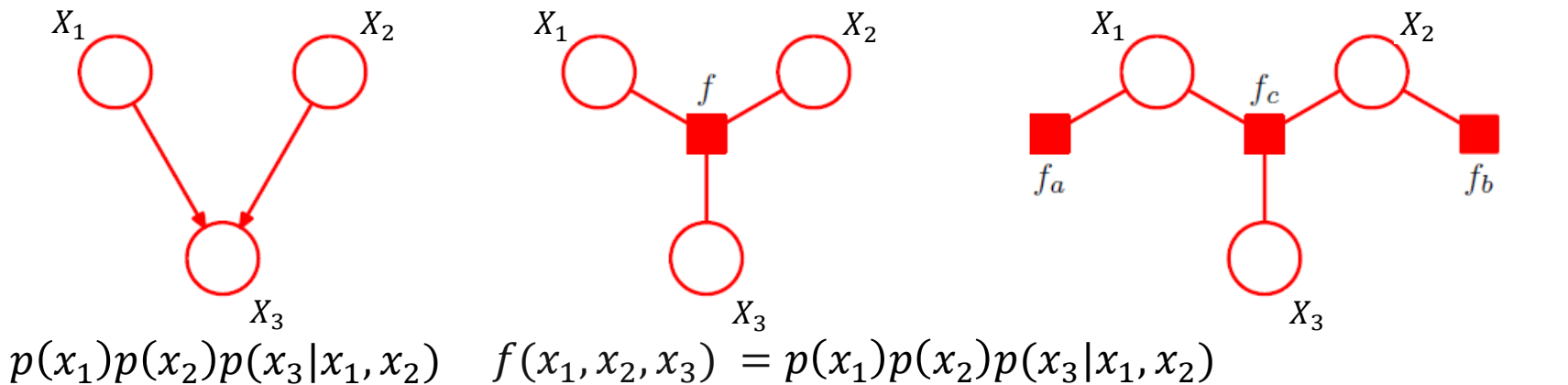
- Note that there may be **several different factor graphs** that correspond to the same DGM / UGM.
- Factor graphs to be **more specific** about the precise form of the factorization.

Example: Directed Graph

$$f_a(x_1) = p(x_1)$$

$$f_b(x_2) = p(x_2)$$

$$f_c(x_1, x_2, x_3) = p(x_3 | x_2, x_1)$$



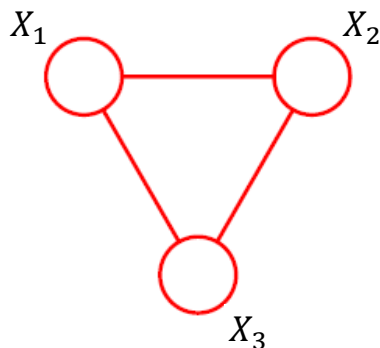
Two factor graphs representing the same distribution

DGM/UGM to Factor Graph

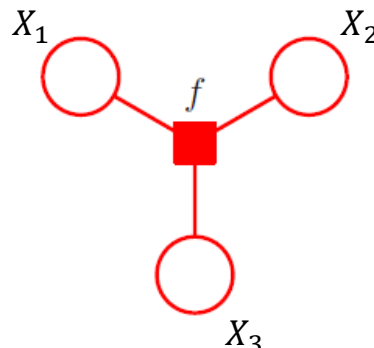
- Note that there may be **several different factor graphs** that correspond to the same DGM / UGM.
- Factor graphs to be **more specific** about the precise form of the factorization.

Example: Undirected Graph

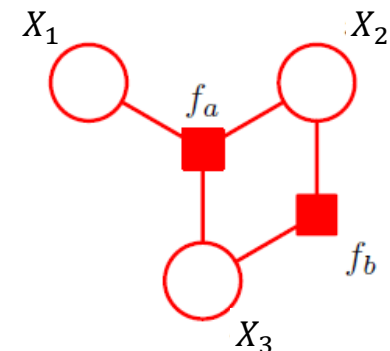
$$f_a(x_1, x_2, x_3)f_b(x_2, x_3) = \psi(x_1, x_2, x_3)$$



Single clique potential
 $\psi(x_1, x_2, x_3)$



$$f(x_1, x_2, x_3) = \psi(x_1, x_2, x_3)$$

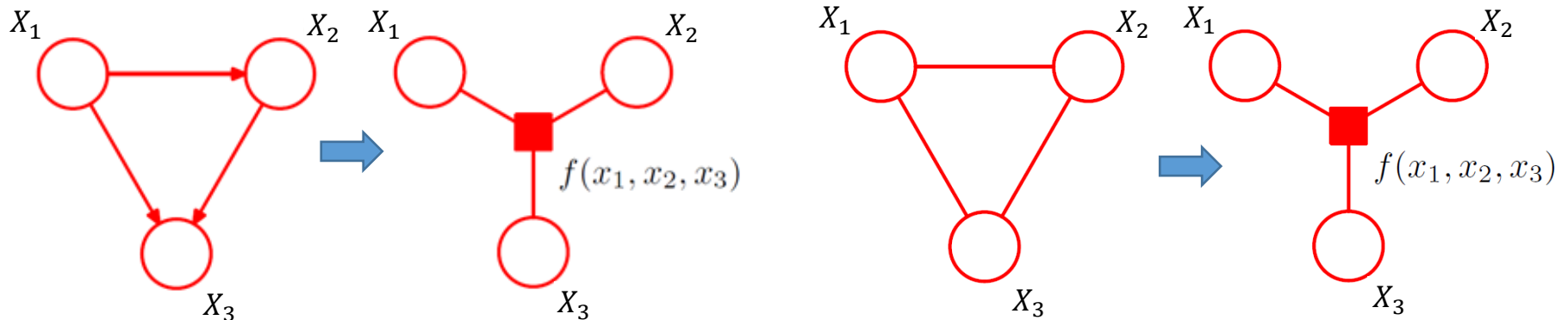


Two factor graphs representing the same distribution

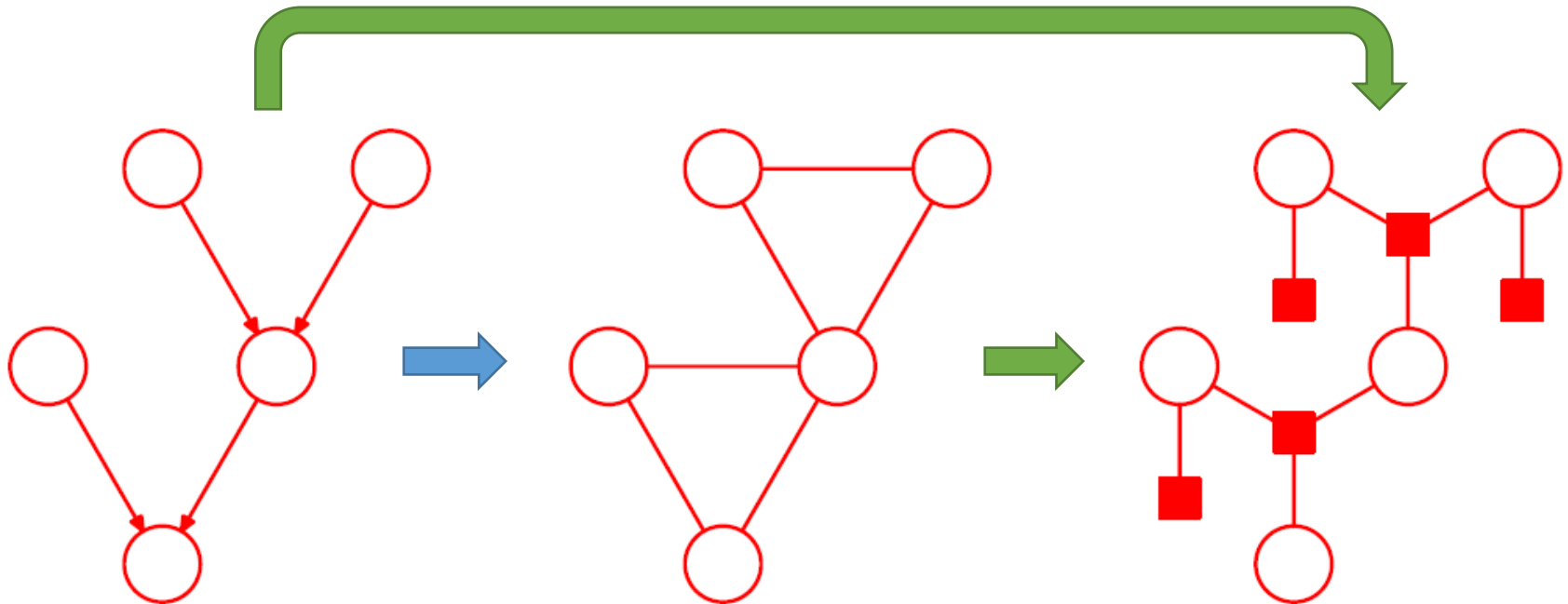
Factor Graphs: Sum-Product Algorithm

- **Alternative representation** for the sum-product algorithm for “tree-like” graphs.
- More importantly, some DGMs/UGMs with local cycles **become a tree** when converted to factor graphs.

Example: Turning local cycle into a tree



Polytrees



- **Cycles appear** after directed to undirected graph conversion.
- **Local cycles disappeared** after factor graph conversion.
- Note the factor graph conversion can be **directly** from a DGM.

Image source: "Pattern recognition and machine learning", Christopher Bishop

Factor Graphs: Sum-Product Algorithm

- **Our goal:** Compute **all singleton marginal probabilities** under the factorized representation of the joint probability.
- As in the earlier **Sum-Product algorithm**, we define two kinds of messages:
 1. Messages ν : flow **from variable to factor nodes**.
 2. Messages μ : flow **from factor to variable nodes**.

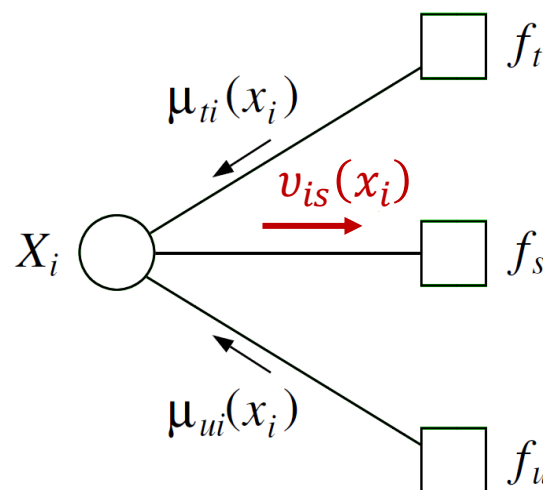
Neighborhood Sets of a Node

- $N(s) \subset \mathcal{V}$: Set of neighbors of a factor node $s \in \mathcal{F}$.
- $N(s)$ refers to the indices of all variables referenced by the factor f_s .
- $N(i) \subset \mathcal{F}$: Set of neighbors of a variable node $i \in \mathcal{V}$.
- $N(i)$ for a variable node X_i refers to the set of all factors that referenced X_i .

Messages from Variable to Factor Nodes

- Message $v_{is}(x_i)$ flows from the **variable node X_i** to the **factor node f_s** :

$$v_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$



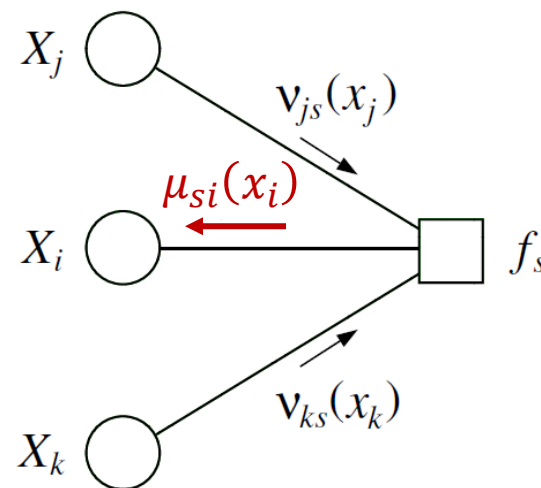
- The product is taken over all incoming messages to the variable node X_i , other than the factor node f_s .

Image source: "An introduction to probabilistic graphical models", Michael I. Jordan, 2002.

Messages from Factor to Variable Nodes

- Message $\mu_{si}(x_i)$ flows from the **factor node f_s** to the **variable node X_i** :

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$

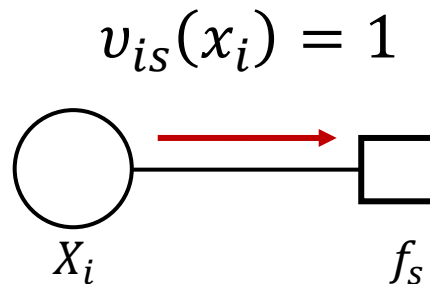


- The product is taken over all incoming messages to the factor node f_s , other than the variable node X_i .

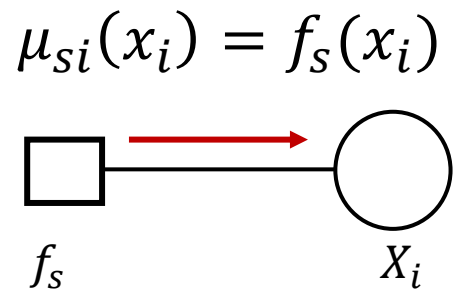
Image source: "An introduction to probabilistic graphical models", Michael I. Jordan, 2002.

Messages From The Leaf Nodes

- Message from a **leaf variable node to factor node**:



- Message from a **leaf factor node to variable node**:



Message-Passing Protocol

A node can send a message to a neighboring node **when (and only when)** it has received messages from all of its other neighbors.

Applies to **both** variable and factor nodes.

Marginal Probability of a Node

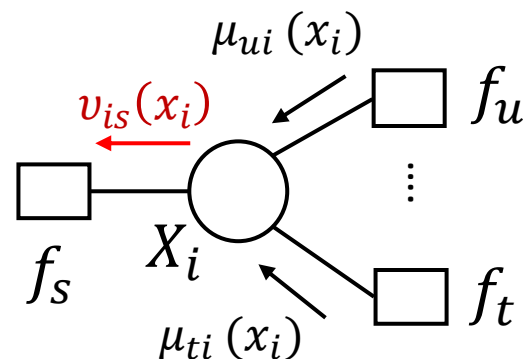
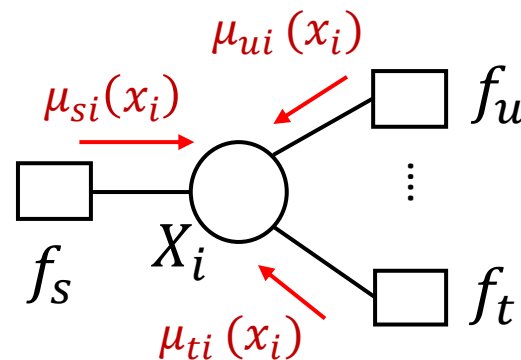
- Once a node X_i has received the messages from all its neighbors, the **marginal probability** is given by:

$$p(x_i) \propto \prod_{s \in \mathcal{N}(i)} \mu_{si}(x_i)$$

$$= \nu_{is}(x_i) \mu_{si}(x_i)$$

since

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$



Factor Tree Sum-Product Algorithm

SUM-PRODUCT($\mathcal{T}, E$) // main steps of **Sum-Product algorithm**

1. EVIDENCE(E)
 $f = \text{CHOOSEROOT}(\mathcal{V})$
2. **for** $s \in \mathcal{N}(f)$
 $\mu\text{-COLLECT}(f, s)$
3. **for** $s \in \mathcal{N}(f)$
 $\nu\text{-DISTRIBUTE}(f, s)$
4. **for** $i \in \mathcal{V}$
 COMPUTEMARGINAL(i)

1. EVIDENCE(E) // add **evidence potentials** (convert conditioning into marginalization)

for $i \in E$
 $\psi^E(x_i) = \psi(x_i)\delta(x_i, \bar{x}_i)$
for $i \notin E$
 $\psi^E(x_i) = \psi(x_i)$

2. $\mu\text{-COLLECT}(i, s)$ // recursively collect messages from leaves to root

for $j \in \mathcal{N}(s) \setminus i$
 $\nu\text{-COLLECT}(s, j)$
 $\mu\text{-SENDMESSAGE}(s, i)$

$\nu\text{-COLLECT}(s, i)$
for $t \in \mathcal{N}(i) \setminus s$
 $\mu\text{-COLLECT}(i, t)$
 $\nu\text{-SENDMESSAGE}(i, s)$

Message from variable node X_i to the factor node f_s :

$\nu\text{-SENDMESSAGE}(i, s)$

$$\prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j)$$

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

Factor Tree Sum-Product Algorithm

SUM-PRODUCT($\mathcal{T}, E$) // main steps of **Sum-Product algorithm**

1. EVIDENCE(E)
 $f = \text{CHOOSEROOT}(\mathcal{V})$
2. **for** $s \in \mathcal{N}(f)$
 $\mu\text{-COLLECT}(f, s)$
3. **for** $s \in \mathcal{N}(f)$
 $\nu\text{-DISTRIBUTE}(f, s)$
4. **for** $i \in \mathcal{V}$
 COMPUTEMARGINAL(i)

1. EVIDENCE(E) // add **evidence potentials** (convert conditioning into marginalization)

for $i \in E$
 $\psi^E(x_i) = \psi(x_i)\delta(x_i, \bar{x}_i)$
for $i \notin E$
 $\psi^E(x_i) = \psi(x_i)$

2. $\mu\text{-COLLECT}(i, s)$ // recursively collect messages from leaves to root

for $j \in \mathcal{N}(s) \setminus i$

$\nu\text{-COLLECT}(s, j)$

$\mu\text{-SENDMESSAGE}(s, i)$

Message from factor node f_s to the variable node X_i :

$\mu\text{-SENDMESSAGE}(s, i)$

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$

$\nu\text{-COLLECT}(s, i)$

for $t \in \mathcal{N}(i) \setminus s$

$\mu\text{-COLLECT}(i, t)$

$\nu\text{-SENDMESSAGE}(i, s)$

Message from variable node X_i to the factor node f_s :

$\nu\text{-SENDMESSAGE}(i, s)$

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

Factor Tree Sum-Product Algorithm

SUM-PRODUCT($\mathcal{T}, E$) // main steps of **Sum-Product algorithm**

1. EVIDENCE(E)
 $f = \text{CHOOSEROOT}(\mathcal{V})$
2. **for** $s \in \mathcal{N}(f)$
 $\mu\text{-COLLECT}(f, s)$
3. **for** $s \in \mathcal{N}(f)$
 $\nu\text{-DISTRIBUTE}(f, s)$
4. **for** $i \in \mathcal{V}$
 COMPUTEMARGINAL(i)

3. $\nu\text{-DISTRIBUTE}(i, s)$ // distribute messages from root to leaves

$\nu\text{-SENDMESSAGE}(i, s)$

for $j \in \mathcal{N}(s) \setminus i$

$\mu\text{-DISTRIBUTE}(s, j)$

$\mu\text{-DISTRIBUTE}(s, i)$

$\mu\text{-SENDMESSAGE}(s, i)$

for $t \in \mathcal{N}(i) \setminus s$

$\nu\text{-DISTRIBUTE}(i, t)$

Message from variable node X_i to the factor node f_s :

$\nu\text{-SENDMESSAGE}(i, s)$

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

4. COMPUTEMARGINAL(i) // compute **marginal probability**

$$p(x_i) \propto \nu_{is}(x_i) \mu_{si}(x_i)$$

Factor Tree Sum-Product Algorithm

SUM-PRODUCT($\mathcal{T}, E$) // main steps of **Sum-Product algorithm**

1. EVIDENCE(E)
 $f = \text{CHOOSEROOT}(\mathcal{V})$
2. **for** $s \in \mathcal{N}(f)$
 $\mu\text{-COLLECT}(f, s)$
3. **for** $s \in \mathcal{N}(f)$
 $\nu\text{-DISTRIBUTE}(f, s)$
4. **for** $i \in \mathcal{V}$
 COMPUTEMARGINAL(i)

3. $\nu\text{-DISTRIBUTE}(i, s)$ // distribute messages from root to leaves

$\nu\text{-SENDMESSAGE}(i, s)$

for $j \in \mathcal{N}(s) \setminus i$

$\mu\text{-DISTRIBUTE}(s, j)$

$\mu\text{-DISTRIBUTE}(s, i)$

$\mu\text{-SENDMESSAGE}(s, i)$

for $t \in \mathcal{N}(i) \setminus s$

$\nu\text{-DISTRIBUTE}(i, t)$

Message from variable node X_i to the factor node f_s :

$\nu\text{-SENDMESSAGE}(i, s)$

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

Message from factor node f_s to the variable node X_i :

$\mu\text{-SENDMESSAGE}(s, i)$

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$

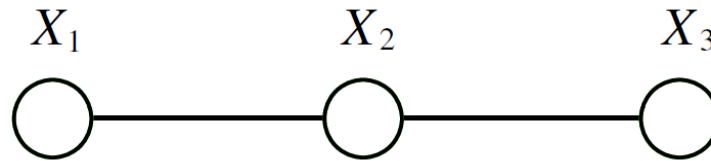
4. COMPUTEMARGINAL(i) // compute **marginal probability**

$$p(x_i) \propto \nu_{is}(x_i) \mu_{si}(x_i)$$

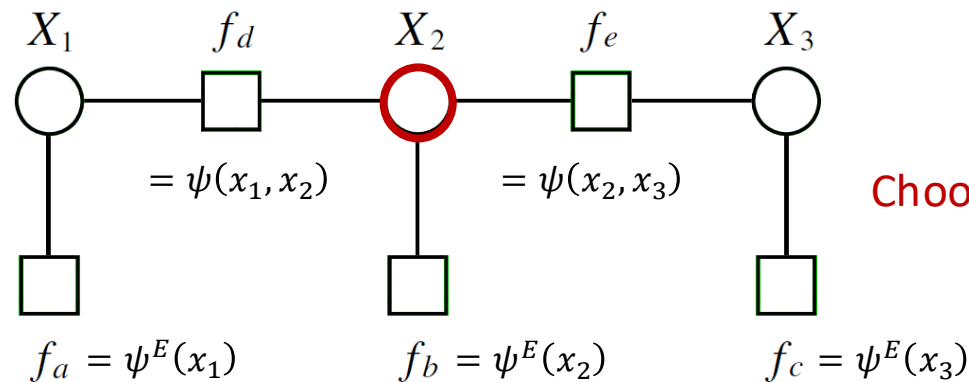
Factor Tree Sum-Product Algorithm

Example:

$$p(x|\bar{x}_E) = \frac{1}{Z^E} (\psi^E(x_1)\psi^E(x_2)\psi^E(x_3)\psi(x_1, x_2)\psi(x_2, x_3))$$



Convert UGM into a factor graph

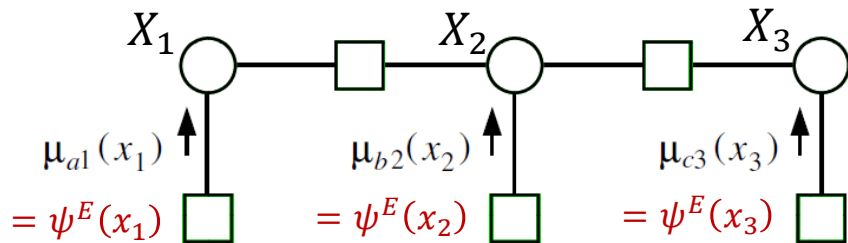


Choose X_2 as root node

Image Source: "An introduction to probabilistic graphical models", Michael I. Jordan, 2002.

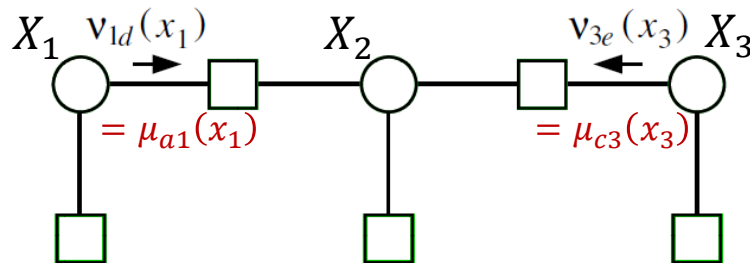
Factor Tree Sum-Product Algorithm

Example:



Collect messages from leaf nodes:

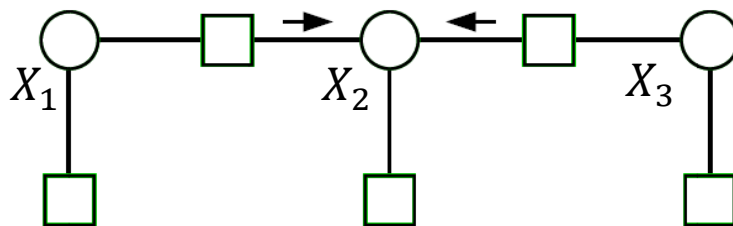
$$\mu_{si}(x_i) = f_s(x_i) = \psi^E(x_i)$$



Collect variable to factor messages:

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

$$\mu_{d2}(x_2) = \sum_{x_1} \psi(x_1, x_2) \mu_{a1}(x_1) \quad \mu_{e2}(x_2) = \sum_{x_3} \psi(x_2, x_3) \mu_{c3}(x_3)$$



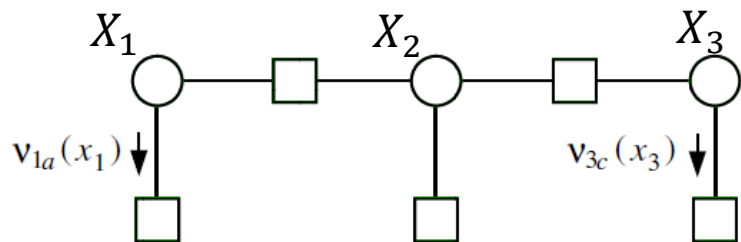
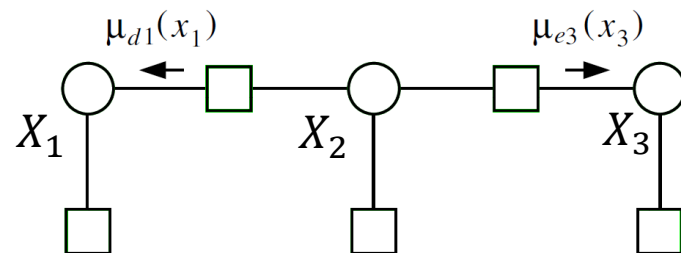
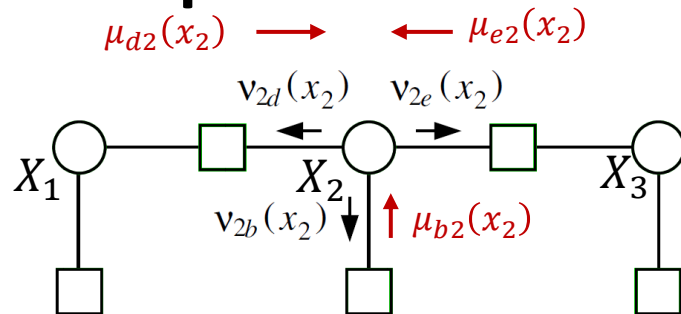
Collect factor to variable messages:

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$

Image Source: "An introduction to probabilistic graphical models", Michael I. Jordan, 2002.

Factor Tree Sum-Product Algorithm

Example:



Distribute variable to factor messages:

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

$$\nu_{2b}(x_2) = \mu_{d2}(x_2) \mu_{e2}(x_2)$$

$$\nu_{2d}(x_2) = \mu_{b2}(x_2) \mu_{e2}(x_2)$$

$$\nu_{2e}(x_2) = \mu_{b2}(x_2) \mu_{d2}(x_2)$$

Distribute factor to variable messages:

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$

$$\mu_{d1}(x_1) = \sum_{x_2} \psi(x_1, x_2) \nu_{2d}(x_2)$$

$$\mu_{e3}(x_3) = \sum_{x_2} \psi(x_2, x_3) \nu_{2e}(x_2)$$

Distribute variable to factor messages:

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

$$\nu_{1a}(x_1) = \mu_{d1}(x_1), \quad \nu_{3c}(x_3) = \mu_{e3}(x_3)$$

Relation Between Sum-Product for UGMs and Factor Graph

- $m_{ji}(x_i)$ in the undirected graph is **equal to** $\mu_{si}(x_i)$ in the factor graph!

Proof:

UGM:

$$m_{ji}(x_i) = \sum_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}(x_j) \right)$$

Factor Graph:

$$\begin{aligned} \mu_{si}(x_i) &= \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right) \\ &= \sum_{x_j} \psi(x_i, x_j) \nu_{js}(x_j) \\ &= \sum_{x_j} \psi(x_i, x_j) \prod_{t \in \mathcal{N}(j) \setminus s} \mu_{tj}(x_j) \\ &= \sum_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{t \in \mathcal{N}'(j) \setminus s} \mu_{tj}(x_j) \right) \end{aligned}$$

$\mathcal{N}'(j)$ denotes the neighbourhood of X_j , omitting the singleton factor node associated with $\psi^E(x_j)$.

Maximum a Posterior Probabilities

- **Marginalization problem**: summing over all configurations of sets of random variables.
- **Maximum a Posterior (MAP) problem**: maximizing over all sets of random variables.
- Two aspects to MAP:
 1. Finding the **maximal probability**.
 2. Finding a **configuration** that achieves the maximal probability.

Maximal Probability

- Given a probability distribution $p(x \mid \bar{x}_E)$, the **maximum a posterior probability** is given by:

$$\begin{aligned}\max_x p(x \mid \bar{x}_E) &= \max_x \frac{p(x, \bar{x}_E)}{p(\bar{x}_E)} \quad \leftarrow \text{Can be removed since we are finding max over } X. \\ &= \max_x p(x, \bar{x}_E) \\ &= \max_x p(x) \delta(x_E, \bar{x}_E) \\ &= \max_x p(x)^E\end{aligned}$$

where

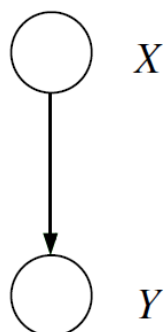
- $\bar{X}_E$ is the set of observed variables, and
- $p(x)^E$ is the unnormalized representation of the conditional probability $p(x \mid \bar{x}_E)$.

Fallacy

- Can we solve the MAP problem by computing the:
 1. marginal probability for **each variable**, and
 2. assignment of each variable that **maximizes its individual marginal**?

NO!!!

Illustration:



1	.6
2	.2
3	.2

$p(x)$

		y		
		1	2	3
x	1	0	.6	.4
	2	1	0	0
	3	1	0	0

$p(y | x)$

	1	2	3
	.4	.36	.24

$p(y)$

Marginal probabilities:

$$\max_x p(x) = p(x = 1) = 0.6$$

$$\max_y p(y) = p(y = 1) = 0.4$$

But

$$\max_{x,y} p(x, y) = p(x = 1, y = 2) = 0.36$$

From Marginal to MAP Algorithms

- **Distributive law** of multiplication over addition:

$$a \cdot b_1 + a \cdot b_2 + \dots + a \cdot b_n = a \cdot (b_1 + b_2 + \dots + b_n)$$

- Plays a **key role** in elimination and sum-product algorithms:

$$\begin{aligned} p(x_1, x_2, \dots, x_5) &= \sum_{x_6} \underbrace{p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)}_a p(x_6|x_2, x_5) \\ &= \sum_{x_6} a \cdot p(x_6|x_2, x_5) \\ &= a \cdot p(x_6 = 0|x_2, x_5) + \dots + a \cdot p(x_6 = k|x_2, x_5) \\ &= a \cdot (p(x_6 = 0|x_2, x_5) + \dots + p(x_6 = k|x_2, x_5)) \quad \text{(Distributive law)} \\ &= p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3) \sum_{x_6} p(x_6|x_2, x_5) \end{aligned}$$

From Marginal to MAP Algorithms

- **Distributive law** applies to the “**max**” operator too!

$$\max(a.b_1, a.b_2, \dots, a.b_n) = a.\max(b_1, b_2, \dots, b_n)$$

- Turn the elimination algorithm into the “**MAP-elimination**” algorithm by replacing the “sum” with “max” operator:

$$\begin{aligned} \max_{x_6} p(x_1, x_2, \dots, x_6) &= \max_{x_6} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5) \\ &\quad \text{“max” operator can be pushed in!} \\ &= \underbrace{p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)}_{\text{independent of } x_6} \max_{x_6} p(x_6|x_2, x_5) \end{aligned}$$

- Becomes the “**max-product**” algorithm.

MAP-Elimination Algorithm

MAP-ELIMINATE($\mathcal{G}, E$) // main steps of the “MAP-Elimination Algorithm”

1. INITIALIZE($\mathcal{G}$)
2. EVIDENCE(E)
3. UPDATE($\mathcal{G}$)
4. MAXIMUM

1. INITIALIZE($\mathcal{G}$) // choose elimination ordering, and add local condition probabilities in **active list**
 choose an ordering I // **same** as the “variable elimination algorithm”
 for each node X_i in $\mathcal{V}$
 place $p(x_i | x_{\pi_i})$ on the active list

2. EVIDENCE(E) // add evidence potentials in **active list**
 for each i in E // **same** as the “variable elimination algorithm”
 place $\delta(x_i, \bar{x}_i)$ on the active list

3. UPDATE($\mathcal{G}$) // **maximization**, and update active list
 for each i in I
 find all potentials from the active list that reference x_i and remove them from the active list
 let $\phi_i^{\max}(x_{T_i})$ denote the product of these potentials
 let $m_i^{\max}(x_{S_i}) = \max_{x_i} \phi_i^{\max}(x_{T_i})$
 place $m_i^{\max}(x_{S_i})$ on the active list

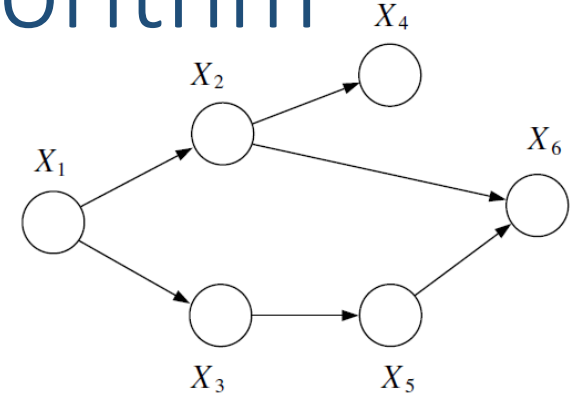
4. MAXIMUM
 $\max_x p^E(x) =$ the scalar value on the active list

MAP-Elimination Algorithm

Example:

Elimination order: $I = \{6, 5, 4, 3, 2, 1\}$

$$p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5)$$



$$\begin{aligned}
 \max_x p(x_1, x_2, x_3, x_4, x_5 | \bar{x}_6) &= \max_{x_1} \max_{x_2} \max_{x_3} \max_{x_4} \max_{x_5} \max_{x_6} \frac{p(x_1, x_2, x_3, x_4, x_5, \bar{x}_6)}{p(\bar{x}_6)} \\
 &= \max_{x_1} \max_{x_2} \max_{x_3} \max_{x_4} \max_{x_5} \max_{x_6} p(x_1, x_2, x_3, x_4, x_5, \bar{x}_6) \\
 &= \max_{x_1} \max_{x_2} \max_{x_3} \max_{x_4} \max_{x_5} \max_{x_6} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5)\delta(x_6, \bar{x}_6) \\
 &= \max_{x_1} p(x_1) \max_{x_2} p(x_2|x_1) \max_{x_3} p(x_3|x_1) \max_{x_4} p(x_4|x_2) \max_{x_5} p(x_5|x_3) \underbrace{\max_{x_6} p(x_6|x_2, x_5)\delta(x_6, \bar{x}_6)}_{m_6^{\max}(x_2, x_5)} \\
 &\quad \underbrace{\hspace{10em}}_{m_5^{\max}(x_2, x_3)} \\
 &\quad \underbrace{\hspace{10em}}_{m_4^{\max}(x_2, x_3)} \\
 &\quad \underbrace{\hspace{10em}}_{m_3^{\max}(x_1, x_2)} \\
 &\quad \underbrace{\hspace{10em}}_{m_2^{\max}(x_1)} \\
 &\quad \underbrace{\hspace{10em}}_{m_1^{\max}(x_1)}
 \end{aligned}$$

Maximal Probability Table

Example: Evidence Node

$x_i \in \{0,1\}$ and we observed that $\bar{X}_6 = 1$

X_2	X_5	X_6	$p(x_6 x_2, x_5)$
0	0	0	v_0
0	0	1	v_1
0	1	0	v_2
0	1	1	v_3
1	0	0	v_4
1	0	1	v_5
1	1	0	v_6
1	1	1	v_7

$\bar{X}_6 = 1$



$$m_6^{max}(x_2, x_5) = \max_{x_6} p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6)$$

X_2	X_5	$m_6^{max}(x_2, x_5)$
0	0	v_1
0	1	v_3
1	0	v_5
1	1	v_7

We are taking a 2d slice of the 3d probabilities or potentials!

Maximal Probability Table

Example:

$$\underbrace{\max_{x_5} p(x_5|x_3)}_{m_5^{max}(x_2, x_3)} \underbrace{\max_{x_6} p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6)}_{m_6^{max}(x_2, x_5)}$$

X_2	X_5	$m_6^{max}(x_2, x_5)$
0	0	v_1
0	1	v_3
1	0	v_5
1	1	v_7

X_3	X_5	$p(x_5 x_3)$
0	0	b_1
0	1	b_2
1	0	b_3
1	1	b_4

X_2	X_3	$m_5^{max}(x_2, x_3)$
0	0	$\max_{x_5} p(x_5 x_3 = 0) m_6^{max}(x_2 = 0, x_5)$ $= \max(p(x_5 = 0 x_3 = 0) m_6^{max}(x_2 = 0, x_5 = 0),$ $\quad p(x_5 = 1 x_3 = 0) m_6^{max}(x_2 = 0, x_5 = 1))$ $= \max(b_1 v_1, b_2 v_3)$
0	1	$\max_{x_5} p(x_5 x_3 = 1) m_6^{max}(x_2 = 0, x_5)$ $= \max(p(x_5 = 0 x_3 = 1) m_6^{max}(x_2 = 0, x_5 = 0),$ $\quad p(x_5 = 1 x_3 = 1) m_6^{max}(x_2 = 0, x_5 = 1))$ $= \max(b_3 v_1, b_4 v_3)$
1	0	$\max_{x_5} p(x_5 x_3 = 0) m_6^{max}(x_2 = 1, x_5)$ $= \max(p(x_5 = 0 x_3 = 0) m_6^{max}(x_2 = 1, x_5 = 0),$ $\quad p(x_5 = 1 x_3 = 0) m_6^{max}(x_2 = 1, x_5 = 1))$ $= \max(b_1 v_5, b_2 v_7)$
1	1	$\max_{x_5} p(x_5 x_3 = 1) m_6^{max}(x_2 = 1, x_5)$ $= \max(p(x_5 = 0 x_3 = 1) m_6^{max}(x_2 = 1, x_5 = 0),$ $\quad p(x_5 = 1 x_3 = 1) m_6^{max}(x_2 = 1, x_5 = 1))$ $= \max(b_3 v_5, b_4 v_7)$

Underflow Problem

- **Products of probabilities** (numbers between 0 and 1) tend to **underflow**!
- Can be overcome by transforming to the **monotone log scale**:

$$\max_x p^E(x) = \max_x \log p^E(x)$$

- Fortunately, the **distributive law** still holds:

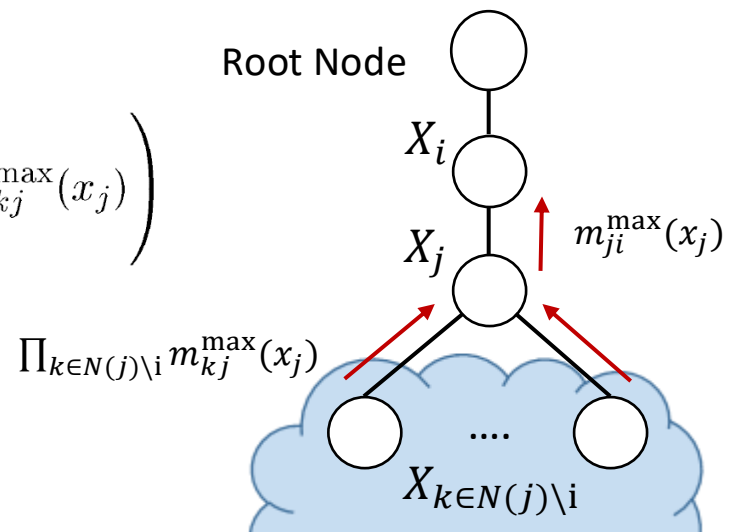
$$\max(a + b_1, a + b_2, \dots, a + b_n) = a + \max(b_1, b_2, \dots, b_n)$$

- Turns the “**max-product**” algorithm into the “**max-sum**” algorithm.

Max-Product Algorithm for Trees

- Find the **MAP probability** for a tree.
- We choose any node X_f as the **root** of the tree, and messages are propagated (inward pass) from the **leaves to the root**.
- Message from X_j to X_i** (closer to root):

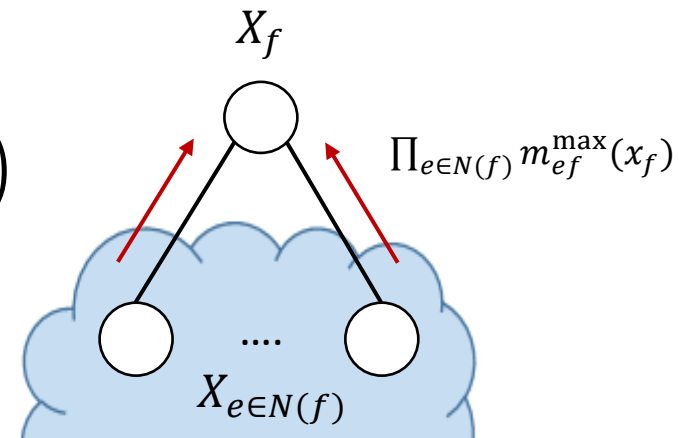
$$m_{ji}^{\max}(x_i) = \max_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j) \right)$$



Max-Product Algorithm for Trees

- Collect all messages at the root and compute the **MAP probability** as:

$$\max_x p^E(x) = \max_{x_f} \left(\psi^E(x_f) \prod_{e \in N(f)} m_{ef}^{\max}(x_f) \right)$$



- Do we need to pass the messages **back to the leaves**?

No!

MAP probabilities for all choices of the root node are **the same**.

Maximum a Posteriori Configurations

- This is the problem of finding **a configuration x^*** such that:

$$x^* \in \operatorname{argmax}_x p^E(x)$$

- Making use of the messages to the root X_f from the **sum-product algorithm**, we obtain a value:

$$x_f^* \in \arg \max_{x_f} \left(\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f) \right)$$

that necessarily belongs to **a maximum configuration**.

Maximum a Posteriori Configurations

- Can we perform an **outward pass** of the messages from the root to leaves so that we can find the MAP configurations for all x ?

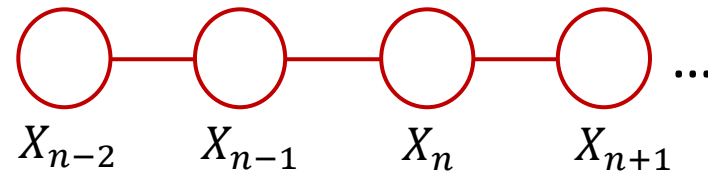
NO!

- **No guarantee** that the values x^* found this way belong to the **same maximizing configuration**.

Maximum a Posteriori Configurations

Example:

A lattice, or trellis, diagram shows two sets of configurations (black paths) in a chain model that give rise to the same MAP probability.



$$x_n \in \{1, 2, 3\}$$

Trellis diagram shows each possible state of the random variable.

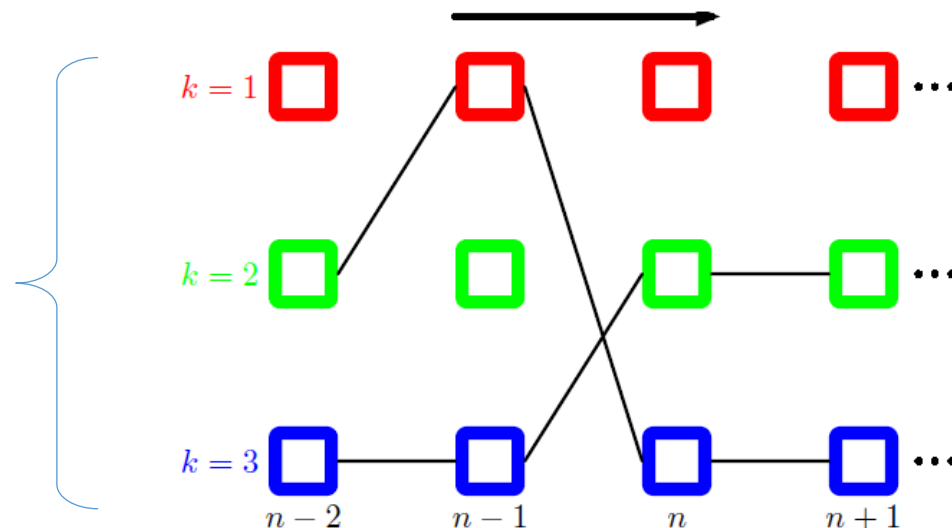


Image source: "Pattern recognition and machine learning", Christopher Bishop

Max-Product Algorithm for Trees

- **Solution:** we also have to **record the maximizing values** in a table $\delta_{ji}(x_i)$ when a message $m_{ji}^{\max}(x_i)$ is sent from X_j to X_i (closer to root):

$$\delta_{ji}(x_i) \in \arg \max_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j) \right)$$

- More precisely, for each X_i , the function $\delta_{ji}(x_i)$ **picks out a value of X_j (can be several)** that achieves the maximum.

Max-Product Algorithm for Trees

- Having defined $\delta_{ji}(x_i)$ during the **inward pass**, we use $\delta_{ji}(x_i)$ to define a consistent maximizing configuration during an **outward pass**:
 1. Choose a maximizing value x_f^* at the root X_f .
 2. Set $x_e^* = \delta_{ef}(x_f^*)$ for each $e \in N(f)$.
 3. Procedure continues outward to the leaves.

Max-Product Algorithm for Trees

MAX-PRODUCT($\mathcal{T}, E$) // main steps of the “MAP-Product Algorithm” for a tree $\mathcal{T}(\mathcal{V}, \mathcal{E})$

EVIDENCE(E)

$f = \text{CHOOSEROOT}(\mathcal{V})$

1. **for** $e \in \mathcal{N}(f)$

 COLLECT(f, e)

$MAP = \max_{x_f} (\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f))$

 // compute MAP probability at root

$x_f^* = \arg \max_{x_f} (\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f))$

 // get MAP configuration at root

2. **for** $e \in \mathcal{N}(f)$

 DISTRIBUTE(f, e)

1. COLLECT(i, j)

 // inward message passing

for $k \in \mathcal{N}(j) \setminus i$

 COLLECT(j, k)

SENDMESSAGE(j, i)

2. DISTRIBUTE(i, j)

 // outward message passing

SETVALUE(i, j)

for $k \in \mathcal{N}(j) \setminus i$

 DISTRIBUTE(j, k)

SENDMESSAGE(j, i)

$m_{ji}^{\max}(x_i) = \max_{x_j} (\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j))$ // compute MAP probability message

$\delta_{ji}(x_i) \in \arg \max_{x_j} (\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j))$ // get MAP configurations

SETVALUE(i, j) // get MAP configuration in outward pass

$x_j^* = \delta_{ji}(x_i^*)$

Max-Product Algorithm for Trees

Example: $x_i \in \{0,1\}$

Inward message passing

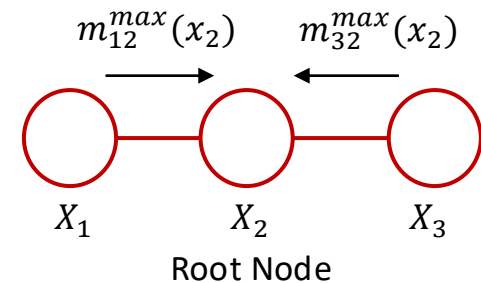
X_2	$m_{12}^{max}(x_2)$	$\delta_{12}(x_2)$
0	$\max_{x_1} \psi(x_1) \psi(x_1, x_2 = 0)$ $= \max(\psi(x_1 = 0) \psi(x_1 = 0, x_2 = 0),$ $\quad \psi(x_1 = 1) \psi(x_1 = 1, x_2 = 0))$ $= \max(a_1, a_2) = a_1$	$x_1^{max} = 0, x_2 = 0$
1	$\max_{x_1} \psi(x_1) \psi(x_1, x_2 = 1)$ $= \max(\psi(x_1 = 0) \psi(x_1 = 0, x_2 = 1),$ $\quad \psi(x_1 = 1) \psi(x_1 = 1, x_2 = 1))$ $= \max(a_3, a_4) = a_4$	$x_1^{max} = 1, x_2 = 1$

*In this example, we assume $a_1 > a_2$ and $a_4 > a_3$.

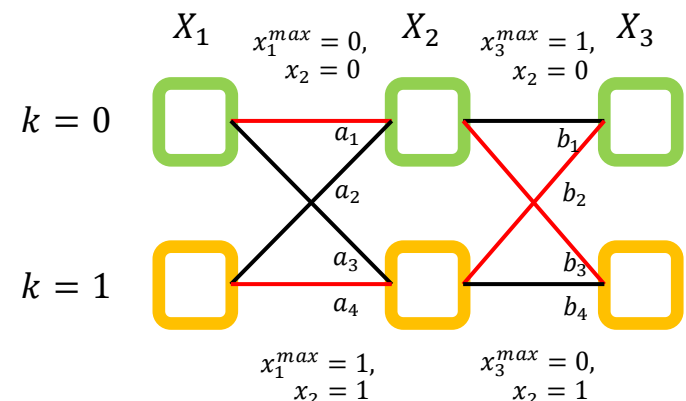
X_2	$m_{32}^{max}(x_2)$	$\delta_{32}(x_2)$
0	$\max_{x_3} \psi(x_3) \psi(x_3, x_2 = 0)$ $= \max(\psi(x_3 = 0) \psi(x_3 = 0, x_2 = 0),$ $\quad \psi(x_3 = 1) \psi(x_3 = 1, x_2 = 0))$ $= \max(b_1, b_3) = b_3$	$x_3^{max} = 1, x_2 = 0$
1	$\max_{x_3} \psi(x_3) \psi(x_3, x_2 = 1)$ $= \max(\psi(x_3 = 0) \psi(x_3 = 0, x_2 = 1),$ $\quad \psi(x_3 = 1) \psi(x_3 = 1, x_2 = 1))$ $= \max(b_2, b_4) = b_2$	$x_3^{max} = 0, x_2 = 1$

*In this example, we assume $b_3 > b_1$ and $b_2 > b_4$.

Find: $\operatorname{argmax}_{x_1, x_2, x_3} p(x_1, x_2, x_3)$



Trellis Diagram:



Max-Product Algorithm for Trees

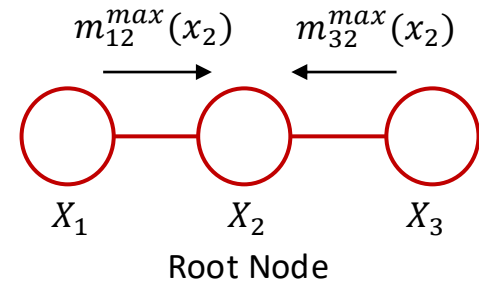
Example: $x_i \in \{0,1\}$

Root node

$m_2^{max}(x_2)$	$\delta_2(x_2)$
$\max_{x_2} \psi(x_2) m_{12}^{max}(x_2) m_{32}^{max}(x_2)$ $= \max(\psi(x_2 = 0) a_1 b_3, \psi(x_2 = 1) a_4 b_2)$ $= \max(d_1, d_2) = d_1 \text{ and } d_2$	$x_2^{max} = 0 \text{ and } 1$

*In this example, we assume $d_1 = d_2$.

Find: $\operatorname{argmax}_{x_1, x_2, x_3} p(x_1, x_2, x_3)$

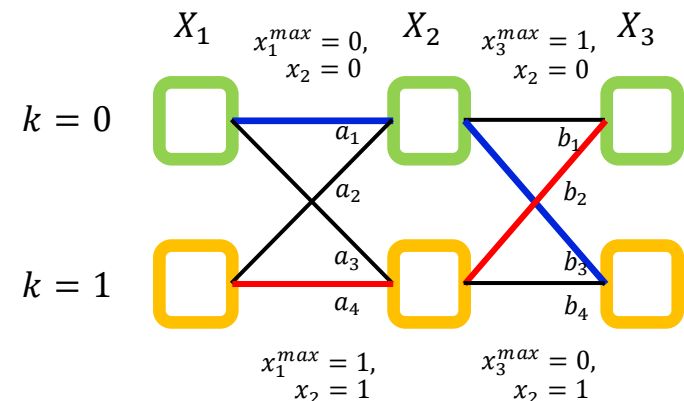


Downward pass

$$\delta_{12}(x_2): x_1^{max} = 0 \leftarrow \delta_2(x_2): x_2^{max} = 0 \rightarrow \delta_{32}(x_2): x_3^{max} = 1$$

$$\delta_{12}(x_2): x_1^{max} = 1 \leftarrow \delta_2(x_2): x_2^{max} = 1 \rightarrow \delta_{32}(x_2): x_3^{max} = 0$$

Trellis Diagram:



From Variable Elimination to Junction Tree

- **Variable Elimination** is query sensitive: we must re-run the entire algorithm for each query node.
- The **Junction Tree algorithm** generalizes Variable Elimination to avoid this.

From Variable Elimination to Junction Tree

- Main idea behind Junction Trees:
 - **Probability distributions** corresponding to loopy undirected graphs can be **re-parameterized as trees**.
 - We can run the **Sum-Product algorithm** on the tree re-parameterization.

Cluster Graphs

- Undirected graph such that:
 1. **Nodes** are **clusters** $C_i \subseteq \{X_1, \dots, X_n\}$, where X_i are the random variables.
 2. **Edge** between C_i and C_j associated with **sepset** $S_{ij} = C_i \cap C_j$.
- A cluster graph organizes overlapping variable clusters and records their shared variables through sepsets.

Cluster Graphs

- **Family preservation:**

Let $\Psi \in \{\psi_1, \dots, \psi_K\}$ be the potentials of the UGM, and define an assignment function:

$$\alpha: \Psi \mapsto \{1, \dots, m\}$$

where $\alpha(\psi) = j$ means that potential ψ is assigned to cluster C_j , i.e. **Scope** $[\psi] \subseteq C_{\alpha(\psi)}$.

- **Cluster potential:**

For cluster C_j , define:

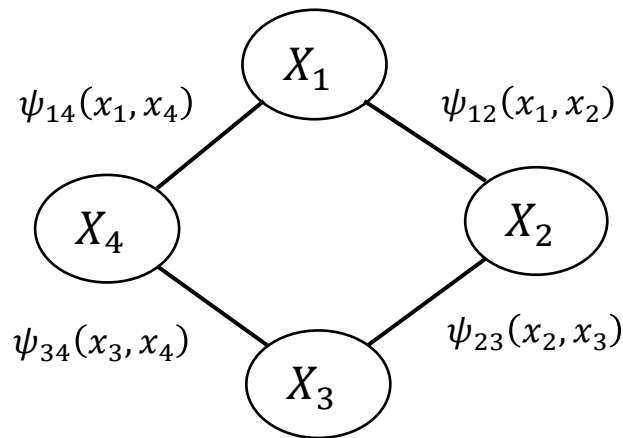
$$\phi_j = \prod_{\psi: \alpha(\psi)=j} \psi, \quad \text{s.t.} \quad \prod_{\psi \in \Psi} \psi = \prod_j \phi_j$$

Each potential is assigned to exactly one cluster, i.e. every ψ is used once.

Cluster Graphs

Example:

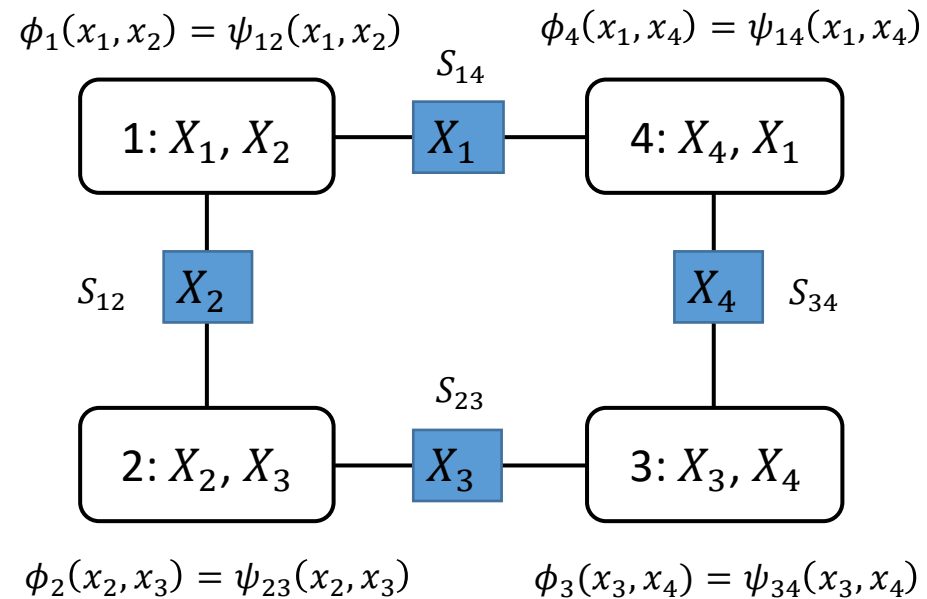
Undirected Graphical Model



Cluster Graph

Sepset: $S_{ij} \subseteq C_i \cap C_j$

Cluster potential: $\phi_i(C_i) = \prod_{k:\alpha(k)=i} \psi_k$



Adapted from: "Probabilistic Graphical Models", Daphne Koller

Running Intersection Property: Junction Tree Property

- For each pair of clusters C_i, C_j and variable $X \in C_i \cap C_j$:

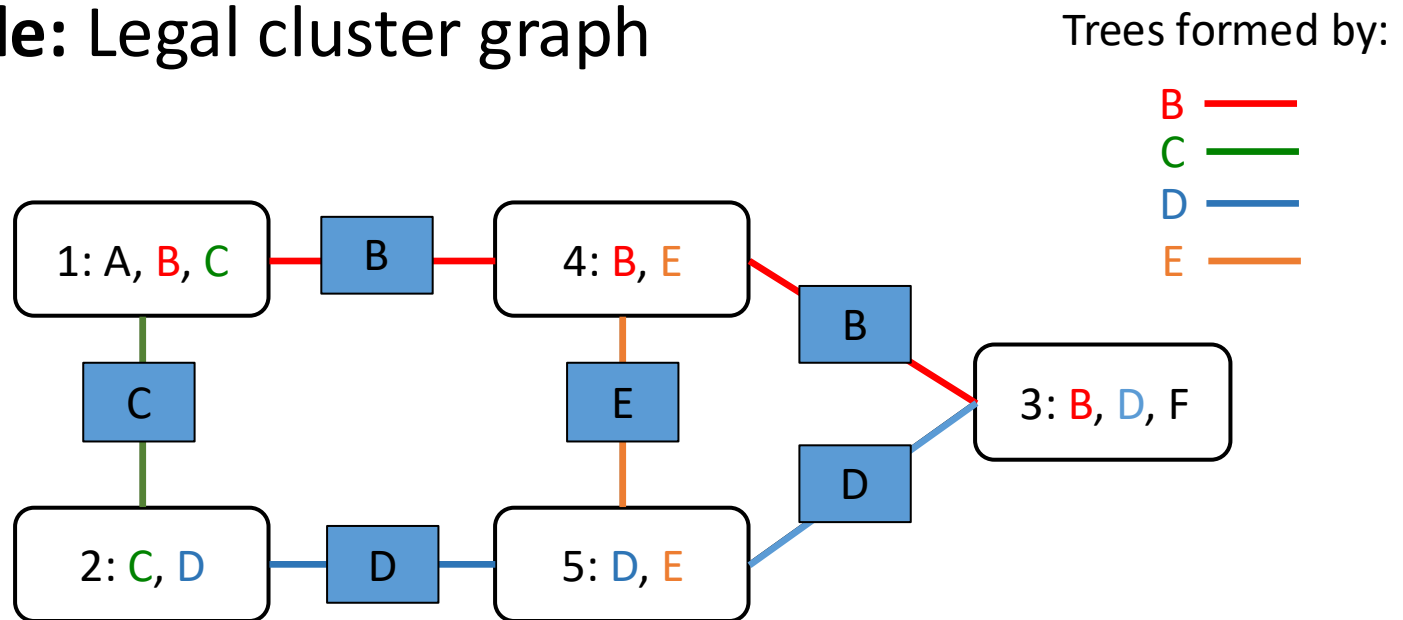
There **exists a unique path** between C_i and C_j for which all clusters and sepsets contain X .

- Equivalently: For any X , the set of clusters and sepsets containing X **form a tree**.

Running Intersection Property: Junction Tree Property

- A valid cluster graph **must fulfil** the running intersection property.

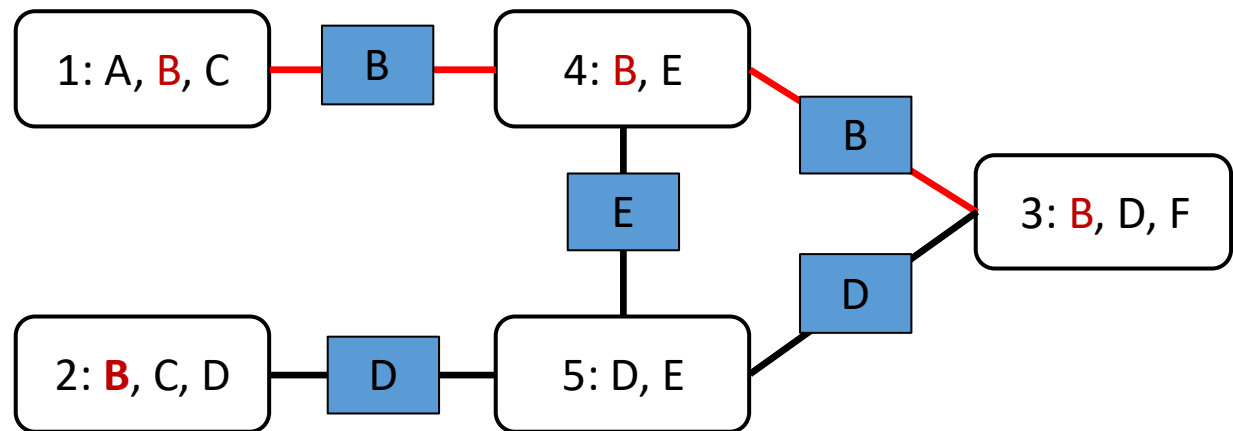
Example: Legal cluster graph



Adapted from: "Probabilistic Graphical Models", Daphne Koller

Running Intersection Property: Junction Tree Property

Example: Illegal cluster graph I

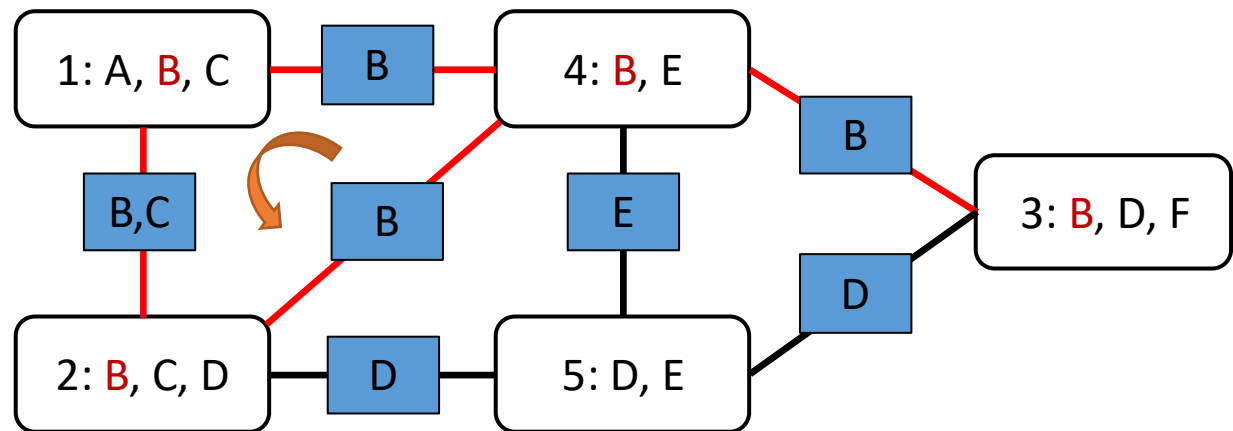


B is **disconnected** from the path!

Adapted from: "Probabilistic Graphical Models", Daphne Koller

Running Intersection Property: Junction Tree Property

Example: Illegal cluster graph II

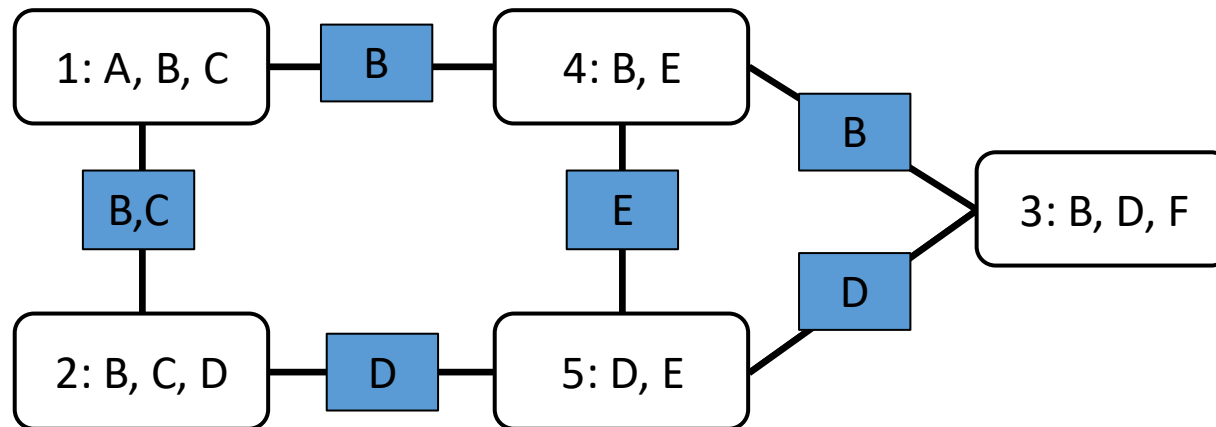


B forms a **cycle**!

Adapted from: "Probabilistic Graphical Models", Daphne Koller

Running Intersection Property: Junction Tree Property

Example: Alternative legal cluster graph



Adapted from: "Probabilistic Graphical Models", Daphne Koller

Clique Trees a.k.a. Junction Trees

- A cluster graph without cycles is known as the **cluster tree**.
- A cluster tree that fulfills the **running intersection property** is called the clique tree, a.k.a. junction tree.
- We refer to a “cluster” in a clique tree as **“clique”**, and “cluster potential” as **“clique potential”**.

Clique Trees a.k.a. Junction Trees

We will first look at how to **compute all marginals** via the junction tree, before looking at how to **convert a DGM/UGM into a junction tree**.

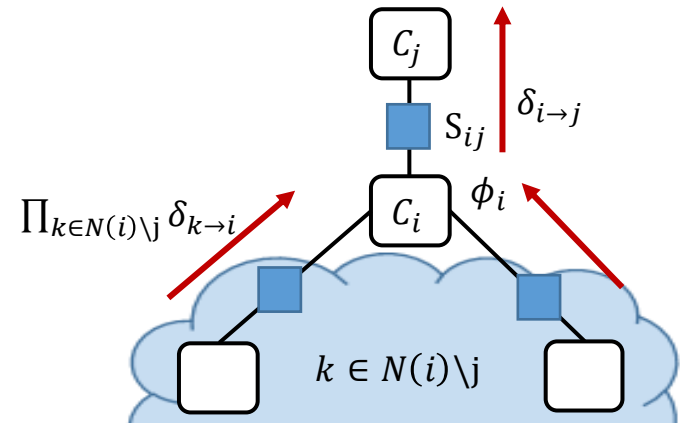
Junction Tree : Sum-Product Algorithm

- We first randomly **choose a root clique**, followed by message passing:
 - **Inward messages** towards the root clique from the leaf cliques.
 - **Outward messages** from the root clique towards the leaf cliques.
- **Message passing protocol**: C_i is ready to pass message to a neighbour C_j when it has received messages from all neighbors except for C_j .

Junction Tree : Sum-Product Algorithm

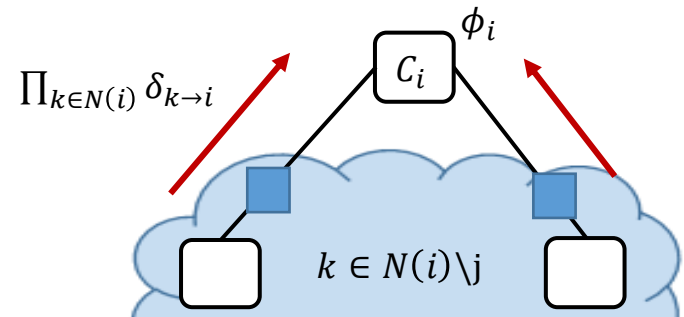
- Use the sum-product algorithm to compute **messages** from C_i to C_j :

$$\delta_{i \rightarrow j} = \sum_{C_i \setminus S_{ij}} \phi_i \cdot \prod_{k \in N(i) \setminus j} \delta_{k \rightarrow i}$$



- The **unnormalized*** marginal probability of clique C_i is given by:

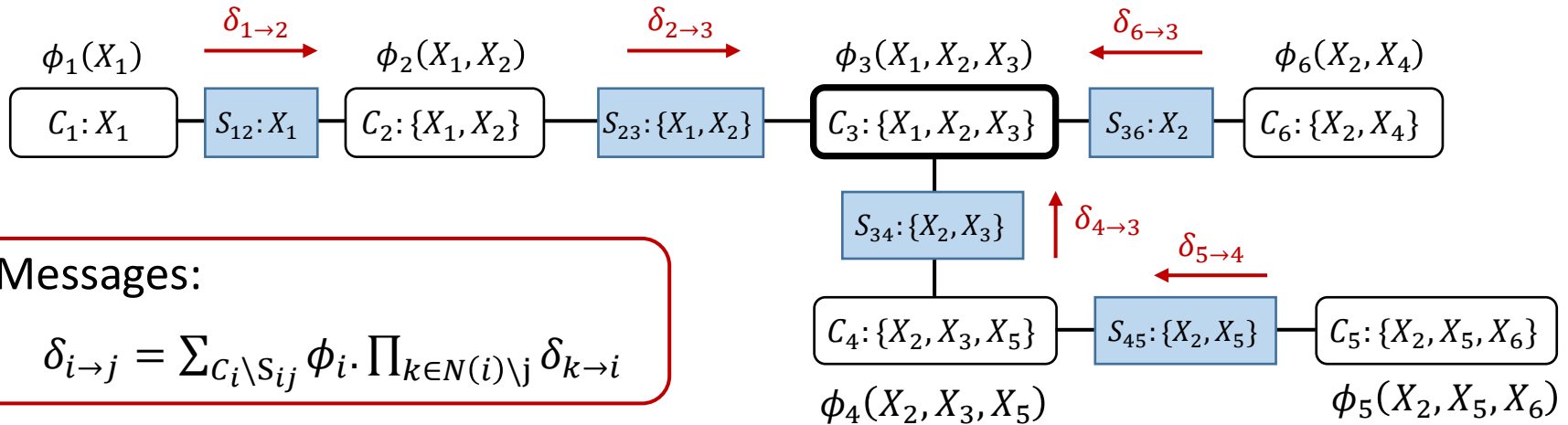
$$\tilde{p}(C_i) = \phi_i \cdot \prod_{k \in N(i)} \delta_{k \rightarrow i}$$



*Unnormalized probability because the clique potentials come from the UGM potentials, where we ignored the partition function

Junction Tree : Sum-Product Algorithm

Example: Let's choose C_3 as the root



Messages:

$$\delta_{i \rightarrow j} = \sum_{C_i \setminus S_{ij}} \phi_i \cdot \prod_{k \in N(i) \setminus j} \delta_{k \rightarrow i}$$

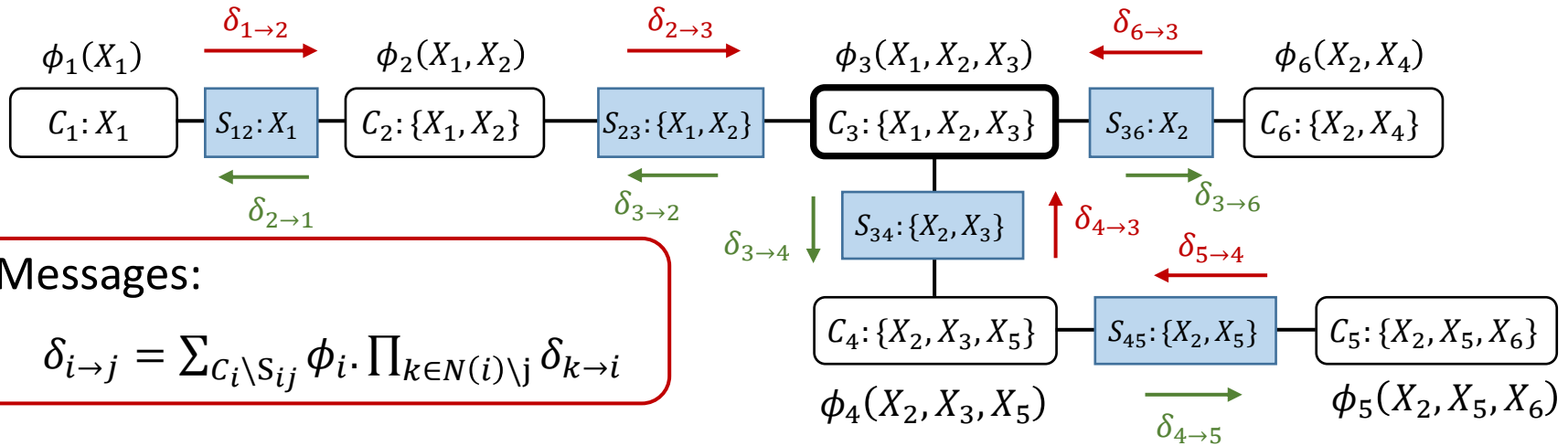
Inward pass:

$$\delta_{1 \rightarrow 2} = \sum_{C_1 \setminus S_{12}} \phi_1 = \phi_1; \quad \delta_{5 \rightarrow 4} = \sum_{C_5 \setminus S_{45}} \phi_5 = \sum_{X_6} \phi_5; \quad \delta_{6 \rightarrow 3} = \sum_{C_6 \setminus S_{36}} \phi_6 = \sum_{X_4} \phi_6$$

$$\delta_{2 \rightarrow 3} = \sum_{C_2 \setminus S_{23}} \phi_2 \cdot \delta_{1 \rightarrow 2} = \phi_2 \cdot \phi_1; \quad \delta_{4 \rightarrow 3} = \sum_{C_4 \setminus S_{34}} \phi_4 \cdot \delta_{5 \rightarrow 3} = \sum_{X_5} \phi_4 \sum_{X_6} \phi_5$$

Junction Tree : Sum-Product Algorithm

Example: Let's choose C_3 as the root



Inward pass:

$$\delta_{1 \rightarrow 2} = \sum_{C_1 \setminus S_{12}} \phi_1 = \phi_1$$

$$\delta_{5 \rightarrow 4} = \sum_{C_5 \setminus S_{45}} \phi_5 = \sum_{X_6} \phi_5$$

$$\delta_{6 \rightarrow 3} = \sum_{C_6 \setminus S_{36}} \phi_6 = \sum_{X_4} \phi_6$$

$$\delta_{4 \rightarrow 3} = \sum_{C_4 \setminus S_{34}} \phi_4 \cdot \delta_{5 \rightarrow 4} = \sum_{X_5} \phi_4 \sum_{X_6} \phi_5$$

$$\delta_{2 \rightarrow 3} = \sum_{C_2 \setminus S_{23}} \phi_2 \cdot \delta_{1 \rightarrow 2} = \phi_2 \cdot \phi_1$$

Outward pass:

$$\delta_{3 \rightarrow 2} = \sum_{C_3 \setminus S_{23}} \phi_3 \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3}$$

$$\delta_{3 \rightarrow 6} = \sum_{C_3 \setminus S_{36}} \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{4 \rightarrow 3}$$

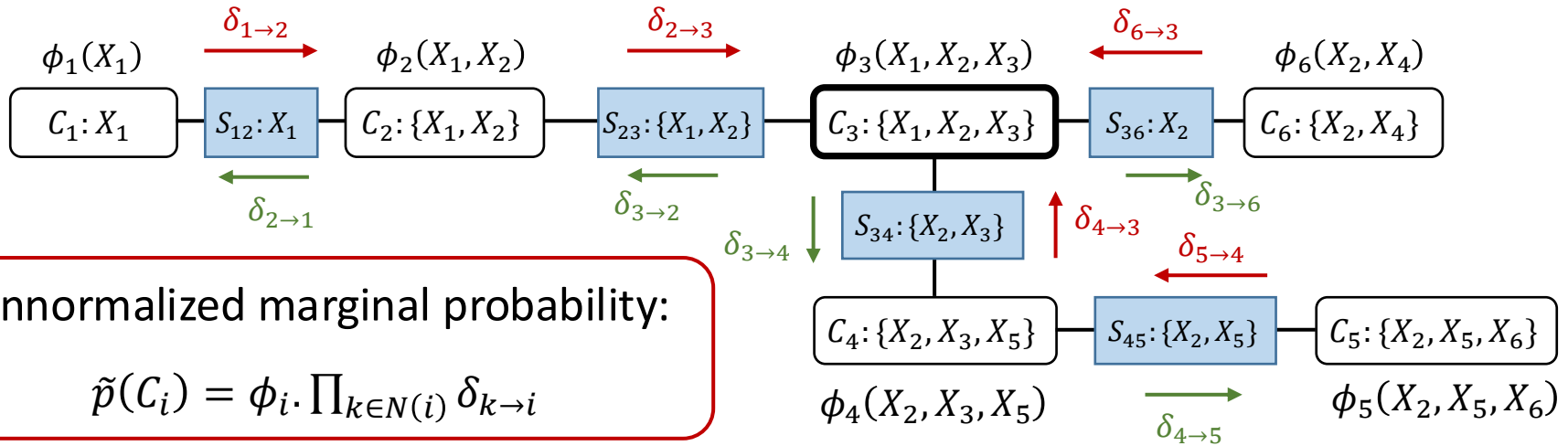
$$\delta_{3 \rightarrow 4} = \sum_{C_3 \setminus S_{34}} \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{6 \rightarrow 3}$$

$$\delta_{2 \rightarrow 1} = \sum_{C_2 \setminus S_{12}} \phi_2 \cdot \delta_{3 \rightarrow 2}$$

$$\delta_{4 \rightarrow 5} = \sum_{C_4 \setminus S_{45}} \phi_4 \cdot \delta_{3 \rightarrow 4}$$

Junction Tree : Sum-Product Algorithm

Example: Let's choose C_3 as the root



$$\begin{aligned}
 \tilde{p}(C_1) &= \tilde{p}(X_1) = \phi_1 \cdot \prod_{k \in N(1)} \delta_{k \rightarrow 1} \\
 &= \phi_1 \cdot \delta_{2 \rightarrow 1} \\
 &= \phi_1 \cdot \sum_{C_2 \setminus S_{12}} \phi_2 \cdot \delta_{3 \rightarrow 2} \\
 &= \phi_1 \cdot \sum_{X_2} \phi_2 \cdot \sum_{C_3 \setminus S_{23}} \phi_3 \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3} \\
 &= \phi_1 \cdot \sum_{X_2} \phi_2 \cdot \sum_{X_3} \phi_3 \cdot \sum_{X_4} \phi_4 \cdot \sum_{X_5} \phi_5 \cdot \sum_{X_6} \phi_6
 \end{aligned}$$

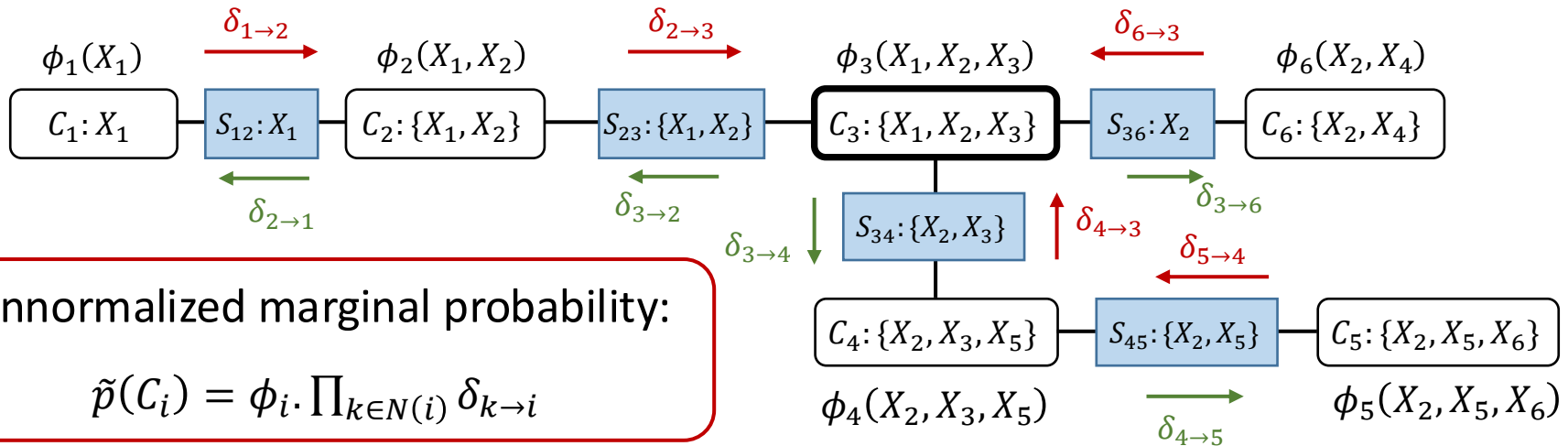
Marginal probability:

$$p(X_1) = \frac{\tilde{p}(X_1)}{\sum_{X_1} \tilde{p}(X_1)}$$

Result is equivalent to variable elimination!

Junction Tree : Sum-Product Algorithm

Example: Let's choose C_3 as the root



$$\tilde{p}(C_2) = \tilde{p}(X_1, X_2)$$

$$= \phi_2 \cdot \prod_{k \in N(2)} \delta_{k \rightarrow 2}$$

$$= \phi_2 \cdot \delta_{1 \rightarrow 2} \cdot \delta_{3 \rightarrow 2}$$

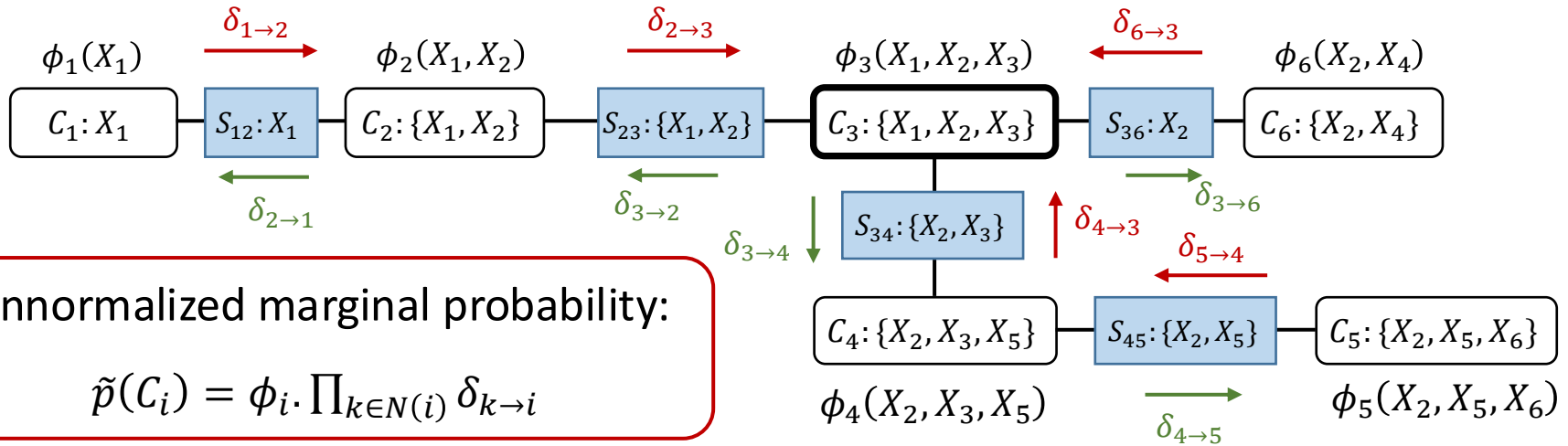
Marginal probabilities:

$$p(X_1, X_2) = \frac{\tilde{p}(X_1, X_2)}{\sum_{X_1} \sum_{X_2} \tilde{p}(X_1, X_2)}$$

$$p(X_2) = \sum_{X_1} p(X_1, X_2)$$

Junction Tree : Sum-Product Algorithm

Example: Let's choose C_3 as the root



$$\begin{aligned} \tilde{p}(C_3) &= \tilde{p}(X_1, X_2, X_3) \\ &= \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3} \end{aligned}$$

$$\begin{aligned} \tilde{p}(C_4) &= \tilde{p}(X_2, X_3, X_5) \\ &= \phi_4 \cdot \delta_{3 \rightarrow 4} \cdot \delta_{5 \rightarrow 4} \end{aligned}$$

$$\begin{aligned} \tilde{p}(C_5) &= \tilde{p}(X_2, X_5, X_6) \\ &= \phi_5 \cdot \delta_{4 \rightarrow 5} \end{aligned}$$

$$\begin{aligned} \tilde{p}(C_6) &= \tilde{p}(X_2, X_4) \\ &= \phi_6 \cdot \delta_{3 \rightarrow 6} \end{aligned}$$

Constructing the Junction Tree

1. **Triangulation**: Get the **reconstituted graph**

Choose an elimination ordering I

DIRECTEDGRAPHELIMINATE(G, I)

1. $G^m = \text{MORALIZE}(G)$ // for DGM, skip this step if UGM
2. $\text{UNDIRECTEDGRAPHELIMINATE}(G^m, I)$ // get reconstituted graph

1. MORALIZE(G)

for each node X_i in I
 connect all of the parents of X_i
end drop the orientation of all edges
return G

2. UNDIRECTEDGRAPHELIMINATE(G, I)

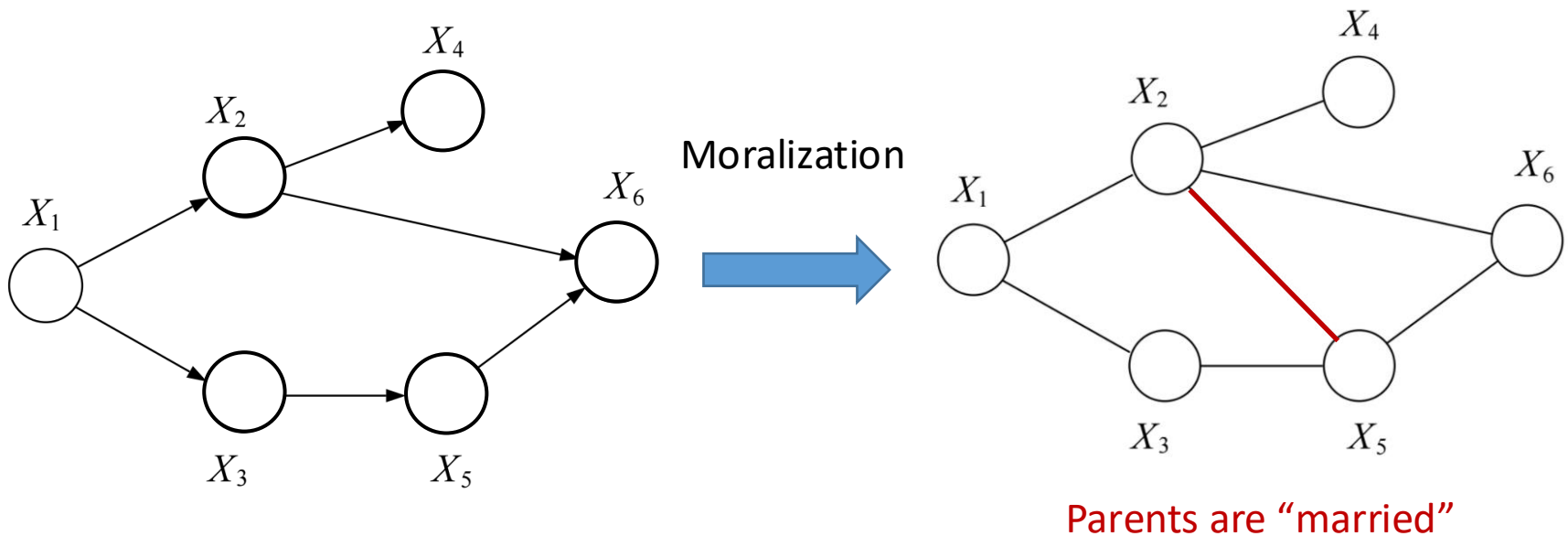
for each node X_i in I
 connect all of the remaining neighbors of X_i
 remove X_i from the graph
end

Source: “An introduction to probabilistic graphical models”, Michael I. Jordan, 2002.

Constructing the Junction Tree

1. **Triangulation**: Get the **reconstituted graph**

Choose an elimination ordering $I = (6; 5; 4; 3; 2; 1)$

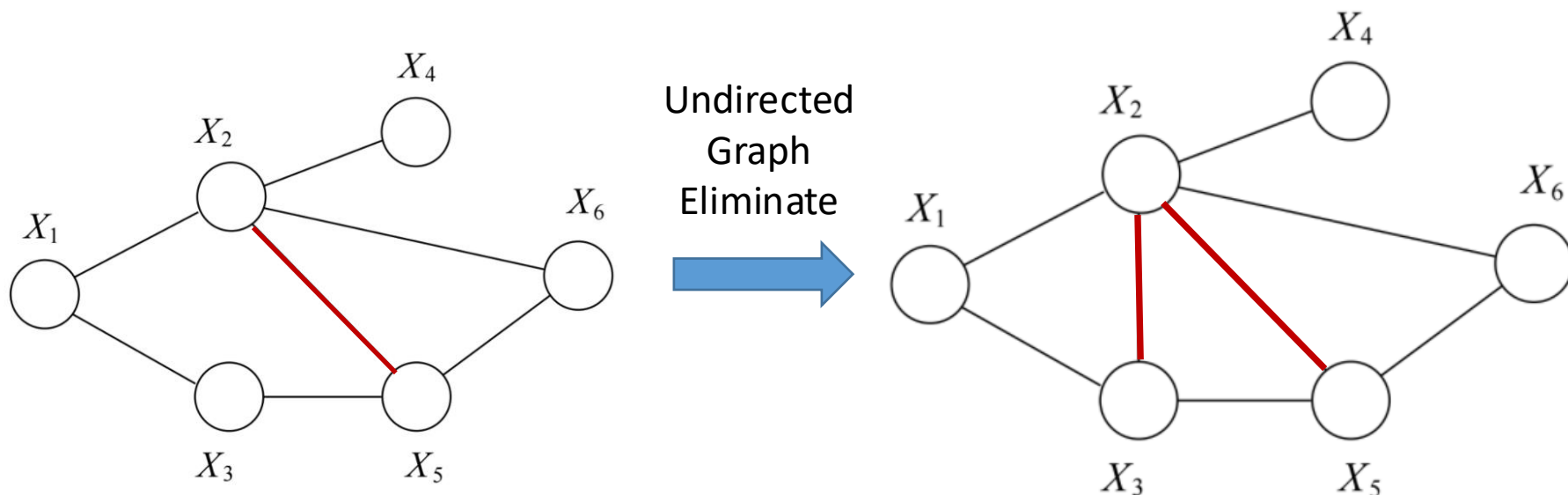


Source: “An introduction to probabilistic graphical models”, Michael I. Jordan, 2002.

Constructing the Junction Tree

1. **Triangulation**: Get the **reconstituted graph**

Choose an elimination ordering $I = (6; 5; 4; 3; 2; 1)$



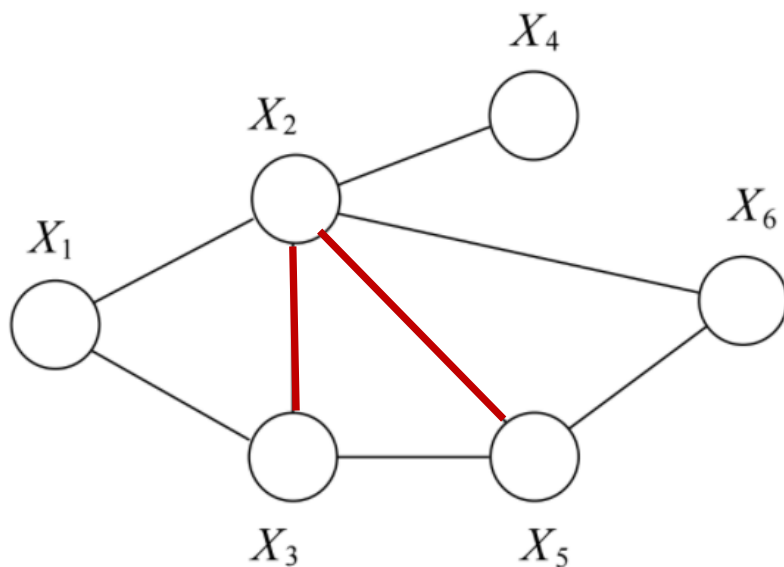
Parents are “married”

Reconstituted graph: additional edges (red) added during the elimination process

Image source: “An introduction to probabilistic graphical models”, Michael I. Jordan, 2002.

Constructing the Junction Tree

2. **Get all clusters and all possible sepsets:** Use eliminate cliques as clusters, a possible sepset is $S_{ij} = C_i \cap C_j$.



$$C_6: \{X_2, X_4\}$$

$$C_5: \{X_2, X_5, X_6\}$$

$$C_4: \{X_2, X_3, X_5\}$$

$$C_3: \{X_1, X_2, X_3\}$$

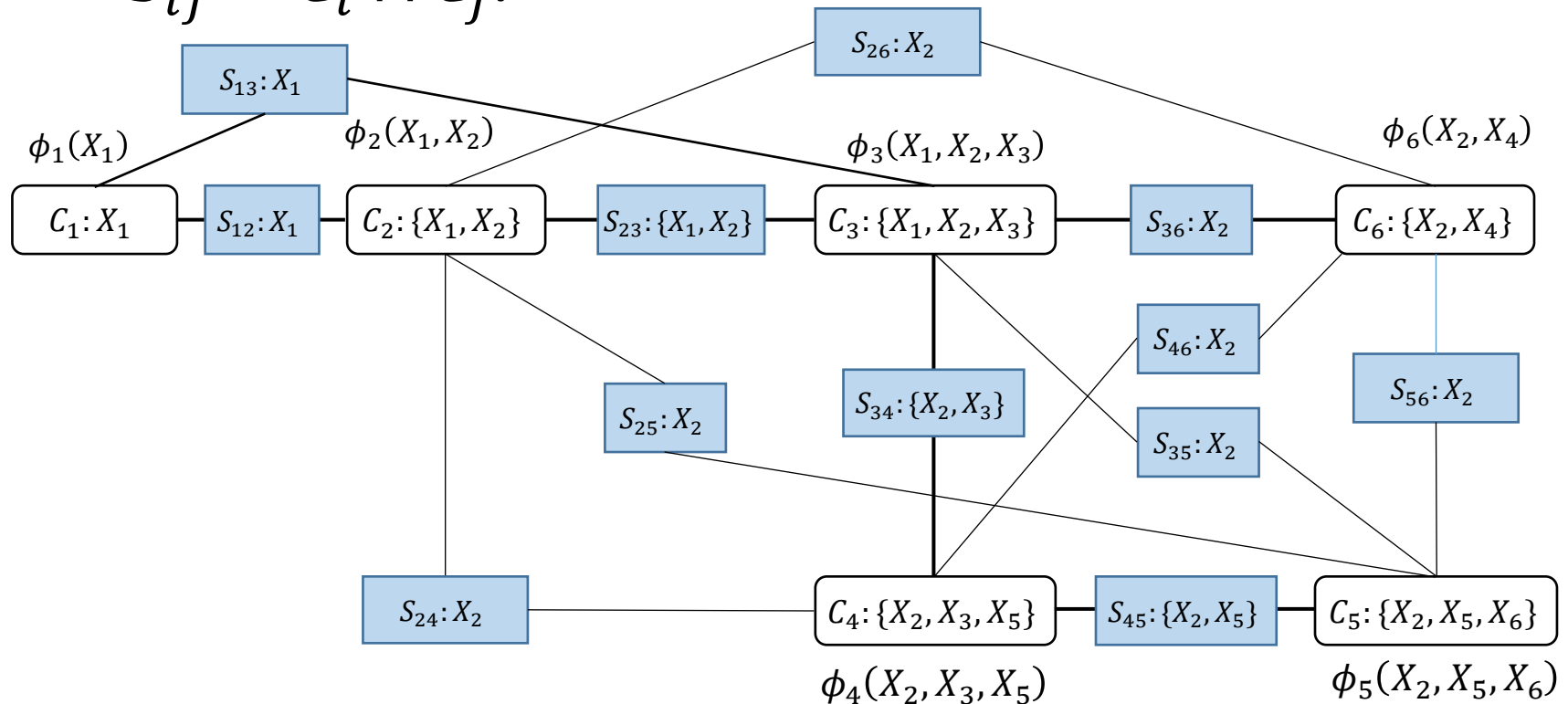
$$C_2: \{X_1, X_2\}$$

$$C_1: X_1$$

Image source: "An introduction to probabilistic graphical models", Michael I. Jordan, 2002.

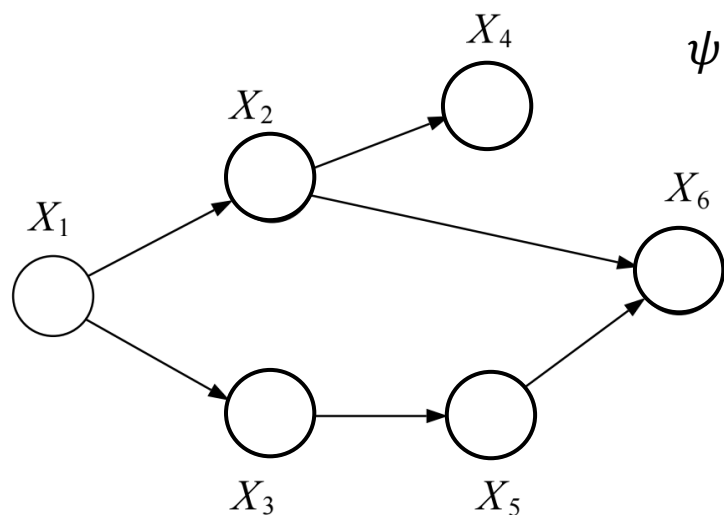
Constructing the Junction Tree

2. **Get all clusters and all possible sepsets:** Use eliminate cliques as clusters, a possible sepset is $S_{ij} = C_i \cap C_j$.



Constructing the Junction Tree

3. **Assign cluster potentials:** cluster potentials are formed by condition probabilities (DGM), or potentials (UGM).



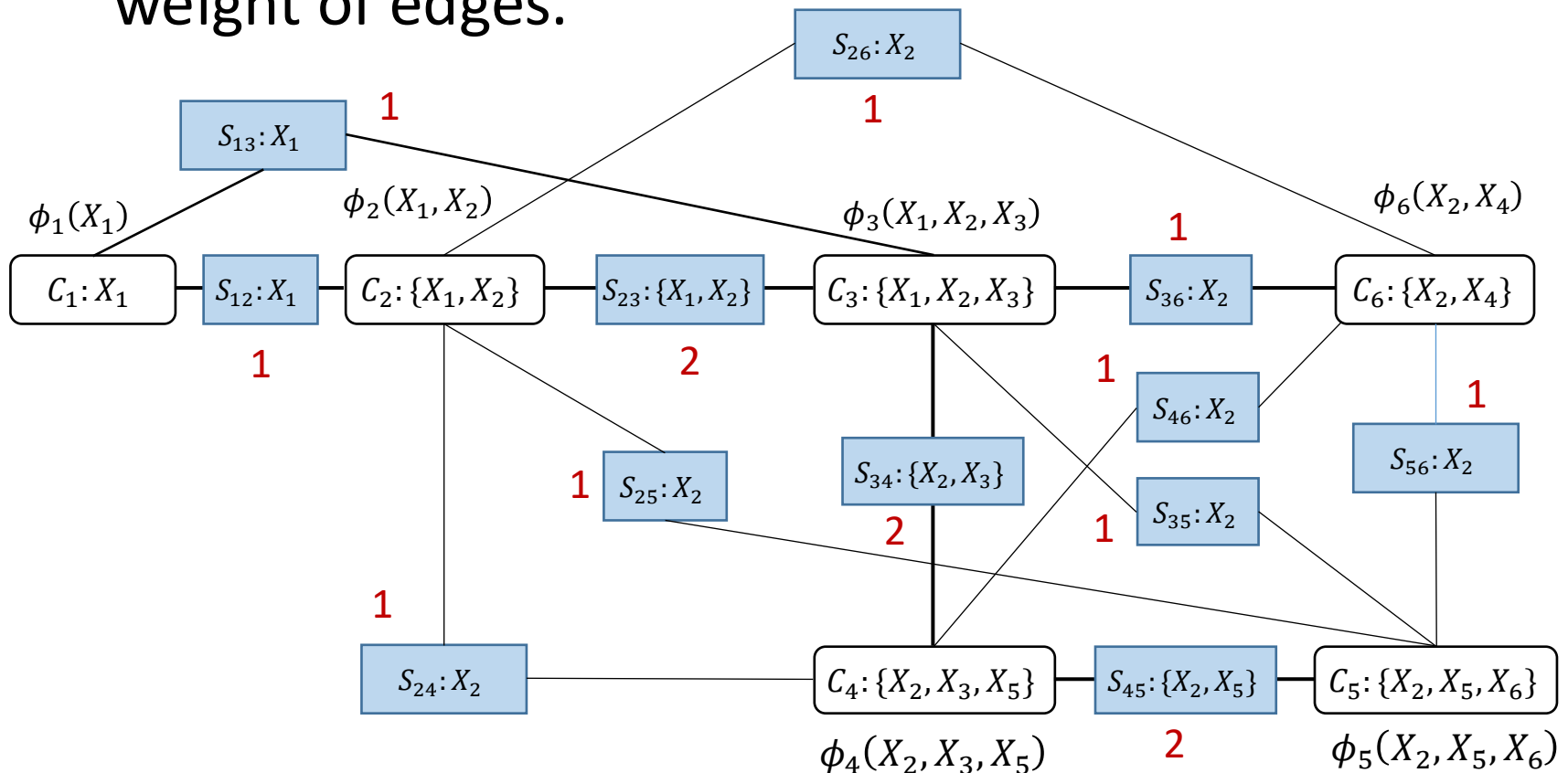
$$p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5)$$
$$\psi(x_1)\psi(x_1, x_2)\psi(x_1, x_3)\psi(x_2, x_4)\psi(x_3, x_5)\psi(x_2, x_5, x_6)$$

Use each conditional probability /
potential only once!

$$\begin{aligned}\phi_1(X_1) &= p(x_1), & \phi_2(X_1, X_2) &= p(x_2|x_1) \\ \phi_3(X_1, X_2, X_3) &= p(x_3|x_1), & \phi_4(X_2, X_3, X_5) &= p(x_5|x_3) \\ \phi_5(X_2, X_5, X_6) &= p(x_6|x_2, x_5), \\ \phi_6(X_2, X_4) &= p(x_4|x_2)\end{aligned}$$

Constructing the Junction Tree

3. **Get clique tree / junction tree**: find the maximum spanning tree with cardinality of sepsets as weight of edges.



Constructing the Junction Tree

3. **Get clique tree / junction tree**: find the **maximum spanning tree** with cardinality of sepsets as weight of edges.

Theorem: A cluster tree T is a clique tree / junction tree only if it is a **maximal spanning tree**.

Constructing the Junction Tree

Proof:

Consider a random variable X_k and a cluster tree T with cluster C_i and sepset S_j , the fact that T is a tree implies:

$1(a)$: **indicator function** that returns 1 if a is true, 0 otherwise

Minus 1 because for tree
#edges = #nodes - 1

$$\sum_{j=1}^{M-1} 1(X_k \in S_j) \leq \sum_{i=1}^M 1(X_k \in C_i) - 1,$$

times X_k appear in the **sepsets**

times X_k appear in the **cluster**

M : # clusters

The inequality sign **becomes equality** when X_k forms a sub-tree, i.e. **running intersection property** is fulfilled.

Constructing the Junction Tree

Proof:

Total weight of a cluster tree $w(T)$ is equal to the sum of the cardinalities of its sepsets:

$$\begin{aligned}
 w(T) &= \sum_{j=1}^{M-1} |S_j| \\
 &= \sum_{j=1}^{M-1} \sum_{k=1}^N 1(X_k \in S_j) \\
 &= \underbrace{\sum_{k=1}^N \sum_{j=1}^{M-1} 1(X_k \in S_j)}_{\text{sum of cardinalities of all sepsets}} \leq \overbrace{\sum_{k=1}^N \left[\sum_{i=1}^M 1(X_k \in C_i) - 1 \right]}^{\text{sum of cardinalities of all clusters minus \# random variables}} \quad \leftarrow \text{From the previous slide} \\
 &= \sum_{i=1}^M \sum_{k=1}^N 1(X_k \in C_i) - N \\
 &= \sum_{i=1}^M |C_i| - N
 \end{aligned}$$

M : # cliques
 N : # random variables

Constructing the Junction Tree

Proof:

$$w(T) = \sum_{j=1}^{M-1} |S_j| \leq \sum_{k=1}^N \left[\sum_{i=1}^M 1(X_k \in C_i) - 1 \right]$$

M : # cliques

N : # random variables

- We saw from previous slide that for the **running intersection property**, i.e. junction tree to hold, the **inequality must become equality**.
- This implies a maximum sum of cardinalities of all sepsets, i.e. **maximal spanning tree**!

□

Constructing the Junction Tree

3. **Get clique tree / junction tree**: find the **maximum spanning tree** with cardinality of sepsets as weight of edges.

```
KRUSKAL(G):  
1  A =  $\emptyset$   
2  foreach v  $\in$  G.V:  
3      MAKE-SET(v)  
4  foreach (u, v) in G.E ordered by weight(u, v), decreasing:  
5      if FIND-SET(u)  $\neq$  FIND-SET(v):  
6          A = A  $\cup$  {(u, v)}  
7          UNION(u, v)  
8  return A
```

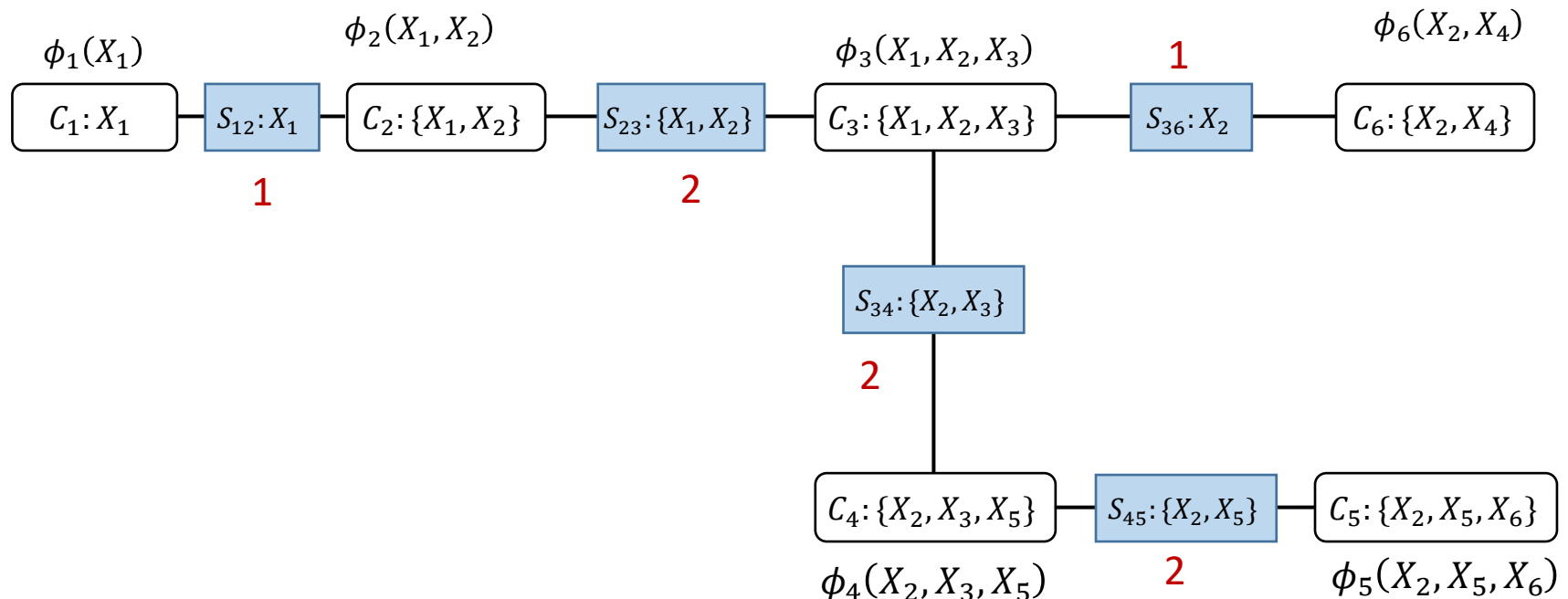
Can be more than 1 maximum spanning tree!

Source: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm

Constructing the Junction Tree

3. **Get clique tree / junction tree**: find the **maximum spanning tree** with cardinality of sepsets as weight of edges.

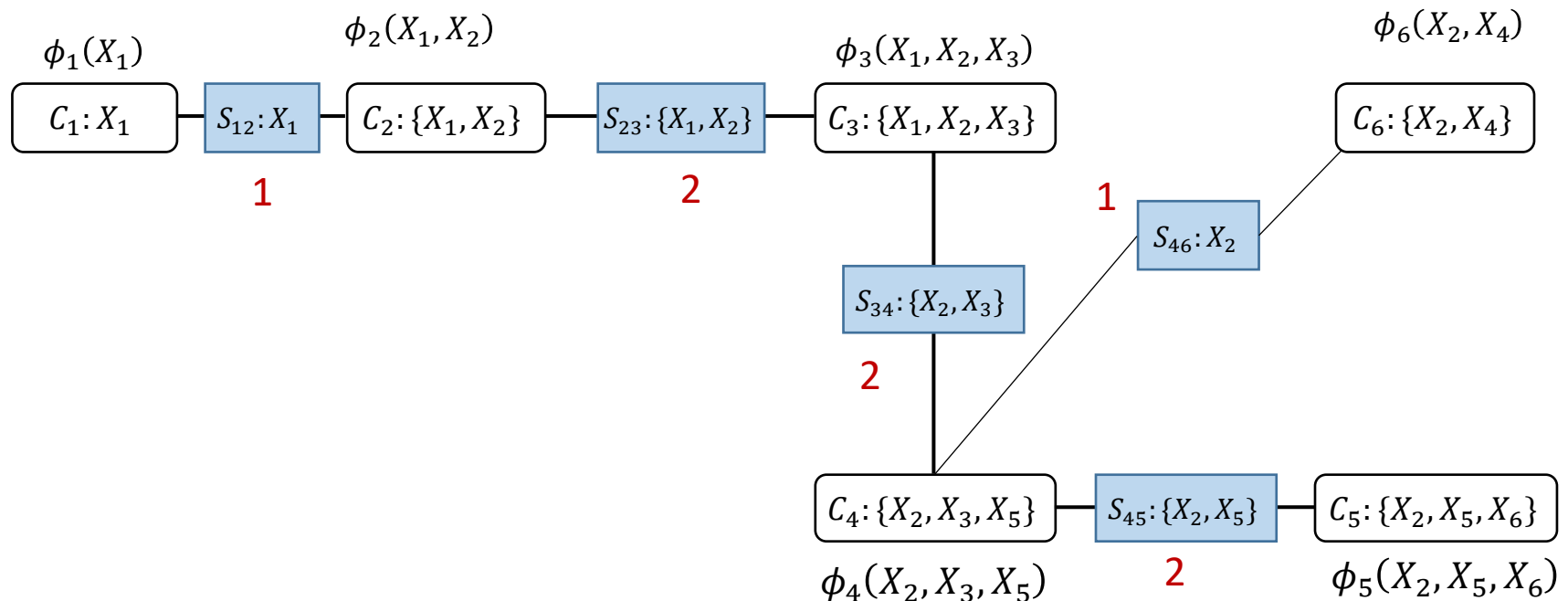
Example:



Constructing the Junction Tree

3. **Get clique tree / junction tree**: find the **maximum spanning tree** with cardinality of sepsets as weight of edges.

Example:



Summary

- We have looked at how to:
 1. Represent a joint distribution with a **factor graph**, and use it to compute the marginal/conditional probabilities.
 2. Use the **max-product algorithm** to find the maximal probability and its configurations.
 3. Convert a DGM/UGM into the **junction tree** and use it to compute the marginal/conditional probabilities.